

---

# **ns-3 Public Safety Communications module**

*Release psc-7.0.1*

**National Institute of Standards and Technology (NIST)**

**Oct 07, 2026**

# CONTENTS

|          |                                              |           |
|----------|----------------------------------------------|-----------|
| <b>1</b> | <b>Public Safety Communications Overview</b> | <b>1</b>  |
| 1.1      | Release History . . . . .                    | 2         |
| 1.2      | Acknowledgments . . . . .                    | 2         |
| <b>2</b> | <b>Mission Critical Push-to-Talk (MCPTT)</b> | <b>3</b>  |
| 2.1      | Model Description . . . . .                  | 3         |
| 2.2      | Design Documentation . . . . .               | 4         |
| 2.3      | Usage . . . . .                              | 11        |
| 2.4      | Testing and Validation . . . . .             | 21        |
| <b>3</b> | <b>UDP Group Echo Server</b>                 | <b>23</b> |
| 3.1      | Model Description . . . . .                  | 23        |
| 3.2      | Usage . . . . .                              | 24        |
| <b>4</b> | <b>UAV Mobility Energy Model</b>             | <b>27</b> |
| 4.1      | Model Description . . . . .                  | 27        |
| 4.2      | Tests . . . . .                              | 28        |
| 4.3      | Usage . . . . .                              | 29        |
| <b>5</b> | <b>Intel HTTP Model</b>                      | <b>30</b> |
| 5.1      | Model Description . . . . .                  | 30        |
| 5.2      | Tests . . . . .                              | 34        |
| 5.3      | Usage . . . . .                              | 35        |
| <b>6</b> | <b>Video Streaming Model</b>                 | <b>36</b> |
| 6.1      | Model Description . . . . .                  | 36        |
| 6.2      | Usage . . . . .                              | 40        |
| <b>7</b> | <b>PSC Application Model</b>                 | <b>41</b> |
| 7.1      | Model Description . . . . .                  | 41        |
| 7.2      | Usage . . . . .                              | 44        |
| <b>8</b> | <b>Incident Scenario Framework</b>           | <b>46</b> |
| 8.1      | Design Overview . . . . .                    | 46        |
| 8.2      | Example: School shooting . . . . .           | 48        |
|          | <b>Bibliography</b>                          | <b>56</b> |

## PUBLIC SAFETY COMMUNICATIONS OVERVIEW

*ns-3* support for public safety communications (PSC) is based on relatively new capabilities for 4G LTE systems introduced in 3GPP Release 12 and later releases. This includes Device-to-Device (D2D) communications over sidelink and mission-critical push-to-talk (MCPTT) application.

Support for public safety communications is distributed among the following five *ns-3* modules:

- 1) `psc`: (this module) Support for models and scenarios that are specific to public safety communications.
- 2) `lte`: Support for ProSe (sidelink communications).
- 3) `buildings`: Support for pathloss models including building effects, as defined by 3GPP with relevance to public safety scenarios.
- 4) `antenna`: Parabolic antenna model as described in 3GPP document TR 36.814.
- 5) `sip`: Session Initiation Protocol (SIP) model.

Documentation for the PSC features implemented in the *ns-3* `lte`, `buildings`, `antenna`, and `sip` modules is provided in the respective module documentation. This chapter documents the *ns-3* `psc` module.

At present, a large portion of the code related to public safety communications is found in the ProSe and UE-to-Network relay implementation in the `lte` module. This is because the ProSe services of sidelink communications, discovery, and synchronization are deeply connected to the LTE models and difficult to factor into a separate module. Features intrinsic to the low-level operation of ProSe in LTE are found in the `lte` module.

The `psc` module also includes:

- 1) an extensive model of mission-critical push-to-talk (MCPTT) for use in off-network or on-network scenarios.
- 2) a UDP-based application to generate many-to-many traffic models in a scenario.
- 3) an energy consumption model for flying a (single/multi)rotor unmanned aerial vehicle (UAV).
- 4) a new HTTP application.
- 5) a video streaming model.
- 6) a configurable generic client/server application
- 7) a framework to develop incident scenarios and a detailed large scale example

There is no support for legacy public safety communications such as land mobile radio system (LMRS). While LTE sidelink supports both IPv4 and IPv6, the UE-to-Network Relay feature only supports IPv6.

## 1.1 Release History

More complete release notes can be viewed also on [GitHub](#).

| Release | Date         | Summary of features                                       |
|---------|--------------|-----------------------------------------------------------|
| v1.0    | Aug 17, 2018 | Release 1.0 based on ns-3.29                              |
| v1.0.1  | Sep 10, 2018 | Minor ns-3.29 release alignment                           |
| v1.0.2  | Sep 17, 2018 | Minor update for Python bindings                          |
| v2.0    | Sep 6, 2019  | Off-network MCPTT, UAV mobility energy, HTTP, discovery   |
| v3.0    | Apr 25, 2020 | LTE UE-to-Network Relay, D2D partial coverage examples    |
| v3.0.1  | Jul 29, 2020 | Aligned to ns-3.31, UE-to-Network Relay fixes             |
| v4.0    | Apr 16, 2021 | On-network MCPTT, video streaming models                  |
| v5.0    | Oct 20, 2021 | Aligned to ns-3.35, large scale scenario and applications |
| v6.0    | May 26, 2022 | Aligned to ns-3.36.1, minor update to MCPTT helper        |
| v7.0    | Jan 4, 2024  | Aligned to ns-3.40                                        |
| v7.0.1  | Oct 8, 2026  | Fix some code robustness and compiler portability issues  |

## 1.2 Acknowledgments

Public safety communications features are based on development led by the Wireless Networks Division of the U.S. National Institute of Standards and Technology (NIST), described in publications ([NIST2016], [NIST2017], [NIST2019], and [NIST2021]). Users of the D2D features of *ns-3* are requested to cite [NIST2017] in academic publications based on these models. Users of the MCPTT features of *ns-3* are requested to cite [NIST2021] in academic publications based on these models.

The integration of the initial ProSe module with the LTE and buildings module, as well as the creation of additional examples and tests, and contributions to on-network MCPTT support, was assisted by CTTC and the University of Washington. This work was performed under the financial assistance award 70NANB17H170 and 70NANB20H179 from U.S. Department of Commerce, National Institute of Standards and Technology.

The following individuals are authors of the public safety communications extensions:

- Zoraze Ali (CTTC)
- Aziza Ben-Mosbah (NIST)
- Evan Black (NIST)
- Fernando J. Cintron (NIST)
- Samantha Gamboa (NIST)
- Wesley Garey (NIST)
- Tom Henderson (Univ. of Washington)
- Antonio Izquierdo Manzanares (NIST)
- Manuel Requena (CTTC)
- Richard Rouil (NIST)
- Raghav Thanigaivel (NIST)

## MISSION CRITICAL PUSH-TO-TALK (MCPTT)

An MCPTT service provides a reliable, always-on, always-available means of communication for first responders. This communication can be one-to-one or one-to-many. There are two modes of operation: on-network and off-network. In on-network mode, communication takes place via a client/server setup, which means that, in LTE, a UE sends signals to an eNodeB, and talks with an MCPTT server via the core-network. But in off-network mode, communication is supported by UE devices in a peer-to-peer like fashion, where signals are only sent from UE to UE.

### 2.1 Model Description

The purpose of the MCPTT model is to enable research into the performance of MCPTT protocols in both off-network and on-network public safety scenarios. Users are expected to be interested in studying Key Performance Indicators (KPIs) such as the push-to-talk access time (KPI1) or the mouth-to-ear latency (KPI3) metrics defined in Section 6.15.3 of [TS22179].

#### 2.1.1 Design

A call is the logical channel between MCPTT applications. Call control is the protocol used to create and manage these logical channels. There are three types of calls that can take place between a set of users: basic, broadcast, and private. A basic group call is one where a logical channel is setup for a group of users associated with a particular group ID so that members of the group can contend to talk in order to communicate with the other members of the group. A broadcast group call is like a basic group call, but only the initiator is allowed to speak, and once the initiator is done speaking the call is terminated. A private call exists when there is a logical channel between two applications for two users to communicate. Each type of call may be basic, imminent peril, or emergency, and this “call status” dictates which physical resources are used by this call.

In the specifications, on-network calls are described not only by their call type (basic, broadcast, and private) but also by how the supporting session is established (on-demand or pre-established), the group call model (pre-arranged or chat group (restricted)), and whether the call requires elevated access privileges to network resources (emergency or imminent peril calls). The mode of call commencement (automatic or manually with user intervention) also can be defined. The difference between an on-demand and pre-established session is that in the pre-established case, certain network setup steps are completed before a user initiates the call; these steps include negotiation of IP addresses and ports (e.g., Interactive Connectivity Establishment (ICE) procedures) and coordination of bearers in the IP Media Subsystem (IMS) core. In a prearranged group call, the group members will be defined a priori, while the chat group (restricted) call model allows members to join calls without being invited. The initial support in *ns-3* for on-network call types is the ‘prearranged group call, using on-demand session, with automatic commencement,’ defined in TS 24.379 [TS24379].

When off-network, all call control messages are sent to all other applications using the same socket. When a user wants to start or join a group call for a particular group ID, the UE sends out periodic “GROUP CALL PROBE” messages. If a UE with same group ID is already part of an on-going call, the UE that received the probe will respond

with a “GROUP CALL ANNOUNCEMENT” message. When a UE who is not part of an on-going call receives this “GROUP CALL ANNOUNCEMENT” message, the UE then automatically joins the call or waits for the user to choose between joining the call or ignoring the call. This announcement message is also sent periodically by members of an on-going call. While part of a group call, floor control (described in the following paragraph) is used to control which UE has permission to send media, and thus, which user is allowed to talk, at any given time during a call. A user may also change the “status” of a call at any point and this is communicated via the “GROUP CALL ANNOUNCEMENT” message.

In an MCPTT group call, only a single member of the group is allowed to talk at a time (with the exception of the dual-speaker feature of on-network), and during this time all other group members need to listen. This is facilitated by the floor control protocol. The protocol consists of floor participants and a floor control server. The floor participants make requests to the floor control server, while the floor control server receives and handles these requests. In the on-network version there is a centralized MCPTT server that acts as the floor control server, but in the off-network case, the UE of the current speaker is the one acting as the floor control server (also known as the current arbitrator). It is optional to use floor control for a private call.

When off-network, all floor control messages are sent to all members of the group. When a user wants to talk, which is indicated by a PTT press during a call, the UE requests permission to send data by sending a “Floor Request” message. The current arbitrator of the call, which is the device of the user who is currently allowed to speak, will send a “Floor Granted” message to give permission to another UE to transmit media, allowing the requesting user to speak to the other members of the group, a “Floor Deny” message to deny the UE permission to send media, or a “Floor Queue Info” message if the UE must wait to transmit media. After sending a request, if the UE does not receive an appropriate message after a given amount of time, the UE will assume that there is currently no arbitrator and will send a “Floor Taken” message, giving itself permission to transmit media.

## 2.2 Design Documentation

The MCPTT model was implemented based on the 3GPP specifications TS 24.379 [TS24379] and TS 24.380 [TS24380] release 14.4.0. This includes off-network models for floor control, basic group calls, broadcast group calls, and private calls. There are also several test cases that were invented by NIST that are also included with this model. This model was created and is intended to be used with the Proximity Services (ProSe) which is provided by LTE.

The MCPTT model includes:

- A set of classes that can be used to model an MCPTT application using call control and floor control in the off-network case
- Helper classes to act as a sink for time-sensitive message and state change traces that can be used to capture the behavior of the different off-network MCPTT state machines (i.e., protocols)
- A helper class that allows users to configure and deploy MCPTT applications in a scenario with nodes
- A few examples that show how the model can be used
- Test cases produced by NIST to check state machine behavior

The MCPTT model does not take into account security. The model only goes as far as including dummy variables in appropriate messages that would carry security fields. Also, the MCPTT model currently does not make use of ProSe Per-Packet Priority (PPPP). Even though the MCPTT model includes and appropriately maintains PPPP as specified in the standards, there currently is no support in ProSe for this feature and, thus, it is not used.

---

**Note:** MCPTT was created specifically for LTE, and the main component that ties off-network MCPTT to LTE is the use of PPPP, which is provided to lower layers of LTE when sending messages for physical resource selection, but the current implementation of the off-network model does not make use of this feature since there is currently no support

for it in the current ns-3 implementation of ProSe. This means that the current MCPTT model is capable of being used over more technologies than just LTE since it is just an application in *ns-3*.

## 2.2.1 Design

In *ns-3*, the `ns3::psc::McpttHelper` is the main class that a user should use to initialize and install applications to an `ns3::Node`. This helper can be used to initially configure some of the components that are associated with an MCPTT application, such as the application's data rate when generating media, the random variable that should be used to simulate a PTT, the IP address used by the application when addressing the group, etc. Once created and installed, it is up to the user to setup calls. This can be done before or after the application is started but not “dynamically” by the application itself. So if a user wanted a scenario with three UE's that will eventually take part in the same on-going group call, the user will have to first query the `ns3::psc::McpttPttApplication` objects of the three `ns3::Node`'s to create those calls. Once the calls are created, the user will also have to query the application to select the current call to reflect things such as a PTT or call release at any given time throughout the simulation.

In the following code snippet, a basic group call is created for N users with group ID 1. One second after the applications are started, the first user will push the PTT button.

```
ObjectFactory callFac;
callFac.SetTypeId(McpttCallMachineGrpBasic::GetTypeId());
callFac.Set("GroupId", UintegerValue(1));
ObjectFactory floorFac;
floorFac.SetTypeId(McpttOffNetworkFloorParticipant::GetTypeId());

for (uint32_t idx = 0; idx < clientApps.GetN(); idx++)
{
    Ptr<McpttPttApp> pttApp = DynamicCast<McpttPttApp, Application>(clientApps.
    ↪Get(idx));
    pttApp->CreateCall(callFac, floorFac);
    pttApp->SelectCall(0);

    if (idx == 0)
    {
        Simulator::Schedule(Seconds(start.GetSeconds() + 1.0),
                            &McpttPttApp::TakePushNotification,
                            pttApp);
    }
}
```

## Identifiers

Several MCPTT identifiers are used throughout the code, as follows.

- **MCPTT User ID** The MCPTT User ID is, in practice, a SIP URI (i.e., a string value). In this ns-3 model, for simplicity, it is stored as a 32-bit unsigned integer. Clients for which the `ns3::psc::McpttHelper` installs an instance of the `ns3::psc::McpttPttApp` will be assigned a unique user ID value, starting from the initial value of 1. In addition, the MCPTT server is separately assigned a user ID, stored in the `ns3::psc::McpttServerCallMachine` instance. This is initialized by default to 0, and typically left at the default value.
- **MCPTT Group ID** The MCPTT Group ID pertains to group calls; it is also, in practice, a SIP URI (i.e., a string value). In this ns-3 model, for simplicity, it is stored as a 32-bit unsigned integer. On clients, the value is stored in the base class `ns3::psc::McpttCallMachineGrp` as an attribute, and similarly, on the server,

it is stored in an attribute in the base class `ns3::psc::McpttServerCallMachineGrp`. The group ID is usually assigned explicitly by the simulation user as part of call definition.

- **SSRC (synchronization source)** This 32-bit identifier is defined for RTP (see RFC 3550) and is, in practice, supposed to be assigned randomly so as to avoid collisions. The SSRC is associated with a media stream component, and the identifier is carried in RTP and floor control messages. In on-network operation, an SSRC is also assigned to the server (included in server-originated floor control messages). The on-network arbitrator, `ns3::psc::McpttOnNetworkFloorArbitrator`, contains an attribute `TxSsrc` that defaults to the value 0. For simplicity, the clients reuse the MCPTT User ID value for SSRC; i.e., all media streams are identified as being from the same source, in the current version of this model.

## MCPTT Application

The `ns3::psc::McpttPttApp` is the core component of the MCPTT model. It is the object used to manage calls and provide an API with functions that would be available to a user like, starting a call, releasing a call, entering an emergency alert, etc. It also houses a few entities to help simulate the behavior of how an MCPTT application may be used. This class also implements the `ns3::psc::McpttMediaSink` interface, which allows for an instance of the `ns3::psc::McpttMediaSrc` class to generate and pass media messages for transmission.

The `ns3::psc::McpttMediaSrc` is simply a class that generates RTP media messages to be sent. It is used by the application to help model data traffic in the network. It is a rather simple traffic model but can be configured to alter data rate and message size.

The `ns3::psc::McpttMsgParser` is a class that reads the header of a packet to determine which message is being received. This class is used by the application as well as in some test cases.

The `ns3::psc::McpttPusher` is class used to simulate the pushing of a button. The application uses this class to simulate a user that is using the PTT button. This model is also simple but allows the user to configure it with a random variable to reflect how often a user may push and release the PTT button.

The `ns3::psc::McpttCall` class simply aggregates the components needed to have an MCPTT call. This includes a channel for both floor control and media messages, and state machines for call control and floor control.

The `ns3::psc::McpttChannel` class is a wrapper around the `ns3::Socket` class to provide an interface similar to the functions required by the specifications, as well as handle opening and closing sockets on the fly. This class is used by the `ns3::psc::McpttCall` and `ns3::psc::McpttPttApp` classes.

The `ns3::psc::McpttCallMachine` is an interface created to handle the different types of calls that are available. This includes functions for starting and leaving calls. There are many subclasses:

- `ns3::psc::McpttCallMachineGrpBasic` for basic group calls
- `ns3::psc::McpttCallMachineGrpBroadcast` for broadcast group calls
- `ns3::psc::McpttCallMachinePrivate` for private calls, and
- `ns3::psc::McpttCallMachineNull` to “turn-off” call control.

Each of the state machines except for the null state machine have several classes associated with each to represent the different states of the state machine. For example, the class `ns3::psc::McpttCallMachineGrpBroadcastStateB1` is a model of the “B1: start-stop” state of the Broadcast call control state machine from the 3GPP standard describing call control.

The `ns3::psc::McpttFloorParticipant` is an interface created to represent the floor control protocol. There are two subclasses:

- `ns3::psc::McpttOffNetworkFloorParticipant` for the floor control protocol
- `ns3::psc::McpttFloorParticipantNull` to “turn-off” floor control

Just like the state machines for call control, the `ns3::McpttOffNetworkFloorParticipant` class has an `ns3::psc::McpttOffNetworkFloorParticipantState` member which is derived by many classes such as `ns3::McpttOffNetworkFloorParticipantStateHasPerm` to represent the different states of the floor control state machine.

The `ns3::psc::McpttEmergAlertMachine` is an interface to represent the state machine used to maintain a user's emergency alert status. This can be found in TS 24.379 [TS24379]. There is currently one subclass:

- `ns3::psc::McpttEmergAlertMachineBasic` for the emergency alert protocol

In the standard, there is one emergency alert machine associated with one user, but in the current implementation there is an emergency alert machine associated with each `ns3::psc::McpttCallMachineGrpBasic` instance. Unlike the previous state machines, there is no “state” class associated with this machine, all the logic is self-contained.

The `ns3::psc::McpttCallTypeMachine` is an interface for the call type machines described in the standard. These state machines exist to maintain the call type (e.g., basic, emergency, etc.) of a call. There are two subclasses:

- `ns3::psc::McpttCallTypeMachineGrpBasic` for a basic group call, and
- `ns3::psc::McpttCallTypeMachinePriv` for a private call.

These state machines are also self-contained and are not associated with any “state” classes.

Most state machines, if not all, have many members that are of the types `ns3::psc::McpttCounter` and `ns3::psc::McpttTimer`. These classes are used to provide an API to mirror the actions that can be requested of counters and timers that are defined throughout the standards. The `ns3::psc::McpttCounter` class provides an interface that can be used as counters are described in the standard with functions like, “Increment” and “Reset”. The `ns3::psc::McpttTimer` class does the same for timers described in the standard with functions like “Start”, “Stop”, and “Restart”. The `ns3::psc::McpttTimer` is simply a wrapper around the `ns3::Timer` class.

The `ns3::psc::McpttMsg` class is a base class for all MCPTT off-network messages. This class is derived from by the `ns3::psc::McpttCallMsg` class for call control messages, the `ns3::psc::McpttFloorMsg` for floor control, and the `ns3::psc::McpttMediaMsg` for RTP media messages. The `ns3::psc::McpttCallMsg` and `ns3::psc::McpttFloorMsg` classes both have many subclasses to represent each floor control or call control message used in the off-network portion of the standard. For example, the `ns3::psc::McpttFloorMsgRequest` class represents the “Floor Request” message described in TS 24.380 [TS24380]. These classes are sent between the MCPTT applications and are consumed by the appropriate state machines throughout the simulation.

As mentioned above, the `ns3::psc::McpttMediaMsg` class is used to represent an RTP media message, but the actual header for an RTP packet is modeled by the `ns3::psc::McpttRtpHeader` class. This class just defines the fields needed for a basic RTP header which is used by the media message.

The `ns3::psc::McpttCallMsgField` and `ns3::psc::McpttFloorMsgField` classes represent call control and floor control message fields, respectively. These classes are also just used as a base and have many child classes. For example, the `ns3::psc::McpttFloorMsgRequest` class, which represents a “Floor Request” message contains a member of type `ns3::psc::McpttFloorMsgFieldUserId` that describes the ID of the MCPTT user making the floor request.

The `ns3::psc::McpttEntityId` class only exist for simulation to associate an ID with various entities described in the standard. One may find that counters, timers, and various states have an ID, and this ID just helps distinguish between multiple objects of the same type.

The `ns3::psc::McpttFloorQueue` class is a wrapper around the `std::vector` class that provides an interface for the floor queue that is described in TS 24.380 [TS24380]. This class is only associated with a floor machine for use when queuing is enabled in floor control. One can enable queuing in floor control for a particular call, simply by setting the capacity of this queue to a size greater than zero.

The `ns3::psc::McpttQueuedUserInfo` class is used by the `ns3::psc::McpttFloorQueue` class to represent the structure of information needed to store for a user when they are placed in the queue, and made to wait to transmit during floor control.

## Pusher Behavior

Supplement to the MCPTT application, a pusher model exists to simulate the PTT behavior for a group of MCPTT users on a call. This model consists of a centralized entity derived from the `ns3::psc::McpttPusherOrchestrator` class that determines the interarrival time (IAT) and duration of PTT events for a set of `ns3::psc::McpttPusher` objects. The value of this model comes from its ability to simulate the PTT behavior for a group based on data that was collected and generalized from real public safety call logs. Using the data from the call logs two main elements were extracted and realized by the model, and they are, PTT and session durations. In essence, a session is a time span when users (i.e., instances of the `ns3::psc::McpttPusher` class) are actively being scheduled to push and release the PTT button, while the PTT duration determines how long a PTT push will last. These durations are generated using instances of the `ns3::EmpiricalRandomVariable` class that interpolate bins taken from the Cumulative Distribution Function (CDF) curves that were computed after parsing the call log data to determine PTT and session durations. While the durations of PTTs and sessions will resemble the actual data more precisely, the IATs of both PTTs and sessions are based on an activity factor that is set by the user. They are computed using instances of the `ns3::ExponentialRandomVariable`, which is assumed to be the distribution of IATs. This means that the following sequence of non-overlapping steps is typical for generating PTT and session events: IAT, duration, IAT, duration, and so on.

The `ns3::psc::McpttPusherOrchestratorInterface` is a base class for the subclasses:

- `ns3::psc::McpttPusherOrchestrator`,
- `ns3::psc::McpttPusherOrchestratorSpurtCdf`,
- `ns3::psc::McpttPusherOrchestratorSessionCdf`, and
- `ns3::psc::McpttPusherOrchestratorContention`.

The class `ns3::psc::McpttPusherOrchestrator` simply selects a random pusher from the set of pushers being orchestrated and uses two instances of an `ns3::RandomVariableStream`. One for deciding push durations and another for deciding the IAT of push events. The `ns3::psc::McpttPusherOrchestratorSpurtCdf` is a wrapper around the `ns3::psc::McpttPusherOrchestrator` class that initializes the random variables used by `ns3::psc::McpttPusherOrchestrator` to match CDFs from the call logs based on the activity factor that was provided. The `ns3::psc::McpttPusherOrchestratorSessionCdf` class is a decorator that simply starts and stops an underlying orchestrator to create sessions based on the CDFs from the call log and the activity factor set by the user. Finally, the `ns3::psc::McpttPusherOrchestratorContention` class is also a decorator that can be used to introduce contention, by using a threshold and an instance of the `ns3::UniformRandomVariable` class to determine if and when an additional PTT event should occur to collide with another active PTT event.

For `ns3::psc::McpttPusherOrchestratorSpurtCdf` the PTT duration is based on the CDF of PTT durations in figure [PTT Duration CDF](#), while the mean value given to the exponential random variable responsible for generating the IAT is computed as:

$$\text{Average PTT IAT} = (\text{Average PTT Duration}) * ((1 / \text{Voice Activity Factor}) - 1)$$

Here, “Average PTT IAT” is the mean value given to the exponential random variable, “Average PTT Duration” is 4.69 (which is the average PTT duration computed from the CDF data), and “Voice Activity Factory” is a value between (0, 1] that is set by the user.

For `ns3::psc::McpttPusherOrchestratorSessionCdf` the session duration is based on the CDF in figure [Session Duration CDF](#), while the mean value given to the exponential random variable responsible for generating session IAT is computed as:

$$\text{Average Session IAT} = (\text{Average Session Duration}) * ((1 / \text{Session Activity Factor}) - 1)$$

Here, “Average Session IAT” is the mean value given to the exponential random variable, “Average Session Duration” is 8.58 (which is the average session duration computed from the CDF data), and “Session Activity Factory” is a value between (0, 1] that is set by the user.

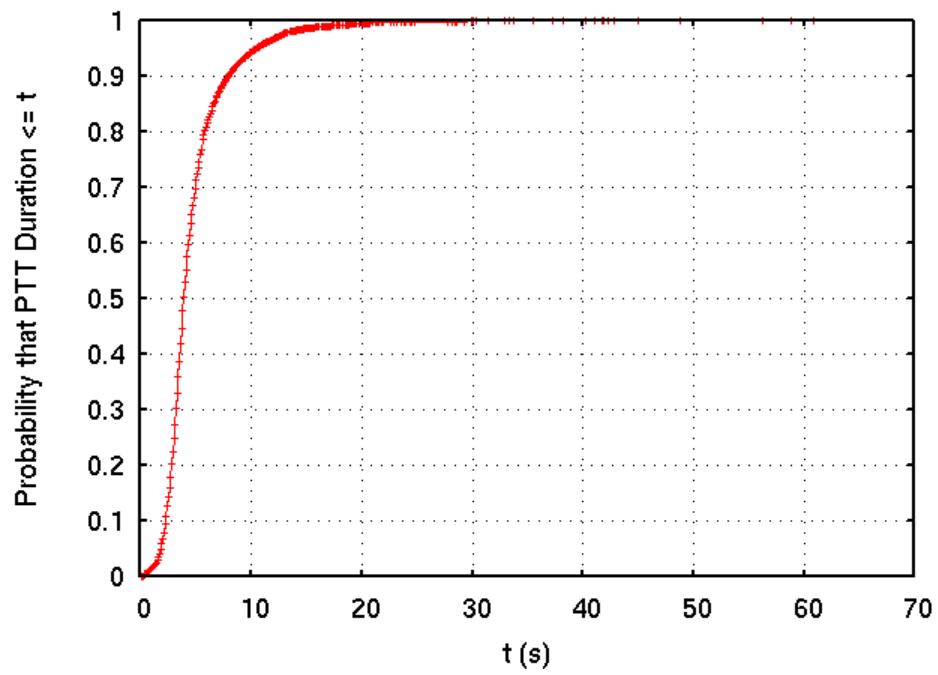


Fig. 1: PTT Duration CDF

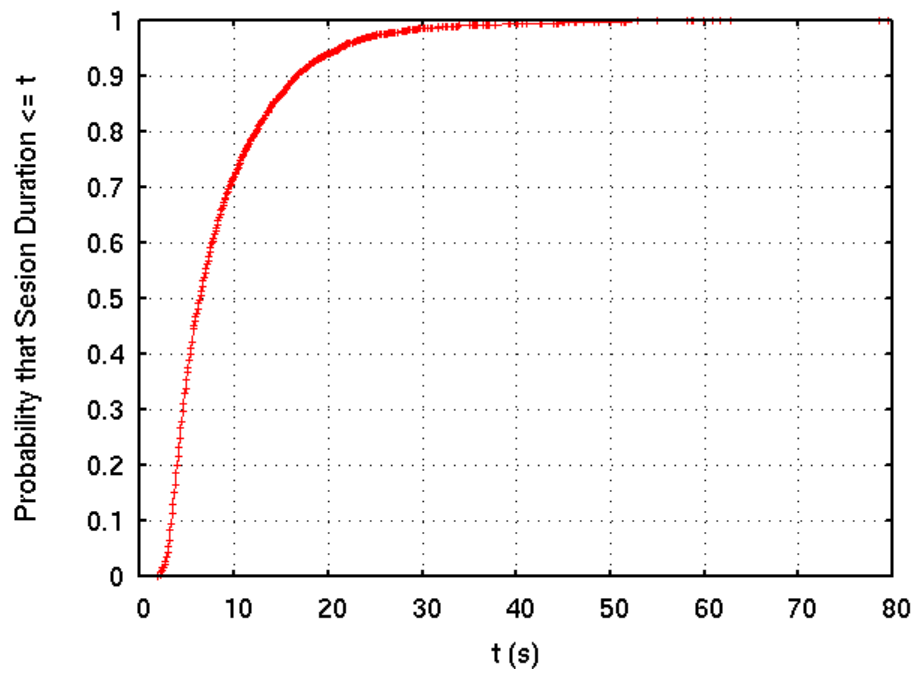


Fig. 2: Session Duration CDF

Note that the `ns3::psc::McpttPusherOrchestratorContention` class can affect the overall activity factor since it can create additional PTT requests, but how it will affect the activity factor depends on the protocol configuration (e.g., queuing, preemption, etc.). However, based on a configurable Contention Probability (CP) threshold, the probability that a single pusher's request will occur at the same time as another pusher's request is:

$$\text{Probability of Experiencing Contention} = 2 * CP / (1 + CP)$$

The following code snippet gives an example of how the pusher model is configured and attached to a set of applications. In this case the voice activity factor during a session is 0.5, which means that there will be a PTT event about 50 % of the time in each session, while the session activity factor is 1.0, which means that there will be active sessions during the entire simulation. The timeline in figure *Pusher Model Example* captures the activity of both pushers and sessions for this configuration.

```
Ptr<McpttPusherOrchestratorSpurtCdf> spurtOrchestrator =
    CreateObject<McpttPusherOrchestratorSpurtCdf>();
spurtOrchestrator->SetAttribute("ActivityFactor", DoubleValue(1.0));

Ptr<McpttPusherOrchestratorSessionCdf> sessionOrchestrator =
    CreateObject<McpttPusherOrchestratorSessionCdf>();
sessionOrchestrator->SetAttribute("ActivityFactor", DoubleValue(0.5));
sessionOrchestrator->SetAttribute("Orchestrator", PointerValue(spurtOrchestrator));

sessionOrchestrator->StartAt(Seconds(2.0));
sessionOrchestrator->StopAt(Seconds(82.0));

mcpttHelper.Associate(sessionOrchestrator, clientApps);
```

It is also worth mentioning that if the Voice Activity Factor is less than 1.0, then depending on the combination of Voice Activity Factor and session duration, it is possible to have a session that does not have any PTT events. This is due to the fact that an active session only determines when PTT events are possible but it does not guarantee that a PTT event will occur.

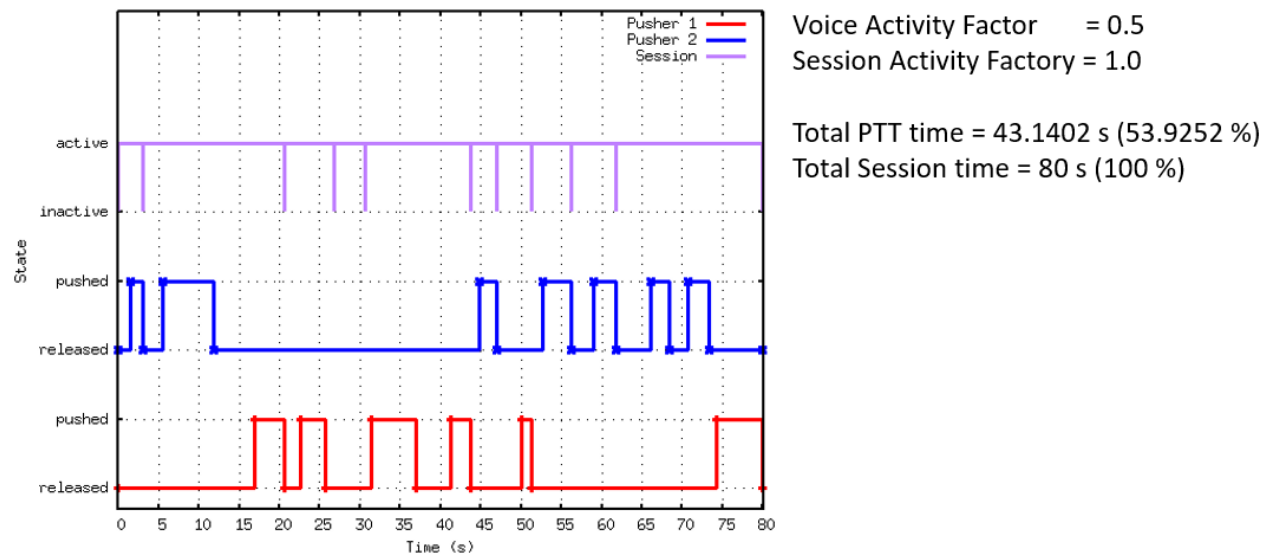


Fig. 3: Pusher Model Example

## Helpers

There are five helpers:

- `ns3::psc::McpttHelper` for deploying MCPTT applications,
- `ns3::psc::McpttMsgStats` for tracing transmitted MCPTT application messages,
- `ns3::psc::McpttProseCollisionDetector` for examining ProSe operation, and
- `ns3::psc::McpttStateMachineStats` for tracing state machine state transitions.
- **`ns3::psc::ImsHelper` for adding an optional IMS;** the IMS is modeled as a single node connected to the PGW.

As stated previously, the `ns3::psc::McpttHelper` is used to configure and deploy MCPTT applications. This is the class that a user should use to configure settings that will be common across the applications and use it to create several copies of the same configuration.

The `ns3::psc::McpttMsgStats` class has the necessary sink functions to connect to and trace messages sent from app to app. One can set the name of the output file by setting the `ns3::psc::McpttMsgStats::OutputFileName` attribute. The user can also specify which types of messages should be captured. So if a user wanted to capture all message types the user should then set, `ns3::psc::McpttMsgStats::CallControl`, `ns3::psc::McpttMsgStats::FloorControl`, and `ns3::psc::McpttMsgStats::Media` attributes to “true”. If the user also wishes to include not just the type of message in the trace but also the contents of the message the user can set `ns3::psc::McpttMsgStats::IncludeMessageContent` to “true” as well.

The `ns3::psc::McpttStateMachineStats` class has the necessary functions to act as a sink to trace state transitions of the various state machines throughout a simulation. One can set the name of the output file by setting the `ns3::psc::McpttStateMachineStats::OutputFileName` attribute.

## 2.2.2 Scope and Limitations

Only the logic of the protocols described for call control and floor control were meant to be captured by the current MCPTT model. There is a feature described throughout the call control document called PPPP or Prose Per-Packet Priority that is taken into consideration at the application but is not propagated to the lower layers as specified because currently this feature is not supported by the ProSe model. On-network currently only supports pre-arranged group call, using an on-demand session, with automatic commencement. Also, elevated call types, such as ‘Emergency,’ and the use of Guaranteed Bit-Rate (GBR) bearers’ are not modeled in on-network as this would require additional features in the LTE model.

## 2.3 Usage

### 2.3.1 Helpers

#### Adding IP Multimedia Subsystem (IMS)

An IMS node can be added to a simulation in a manner similar to the optional configuration of an EPC network. In the below snippet, the `ImsHelper` object is `LteHelper`, and the `ImsHelper` also is connected to the PGW node:

```
Ptr<ImsHelper> imsHelper = CreateObject<ImsHelper>();
imsHelper->ConnectPgw(epcHelper->GetPgwNode());
```

The IMS node itself can be fetched as follows:

```
Ptr<Node> ims = imsHelper->GetImsNode();
```

The `ConnectPgw()` method creates the IMS node, adds an IP (internet) stack to it, and then adds a point-to-point link between it and the PGW node. The SGi interface is assigned the 15.0.0.0/8 network by default, although this can be changed to another network. Finally, a static route towards the UE subnetwork (7.0.0.0/8) is inserted on the IMS.

## 2.3.2 Examples

There are two off-network MCPTT examples in the ‘psc/examples’ folder:

- `example-mcptt.cc` is a basic scenario with two users deployed randomly using the *ns-3* WiFi module in Adhoc mode
- `mcptt-lte-sl-out-of-covrg-comm.cc` is an adaptation of the LTE Sidelink example `lte-sl-out-of-covrg-comm Mode 2 ProSe` example

There are several on-network MCTT examples in the ‘psc/examples’ folder:

- `example-mcptt-on-network-floor-control-csma.cc` demonstrates the operation of the on-network floor control with a single call on a CSMA network.
- `example-mcptt-on-network-floor-control-lte.cc` demonstrates the operation of the on-network floor control with a single call on an LTE network.
- `example-mcptt-on-network-two-calls.cc` demonstrates two sequential calls executing sequentially in the same LTE scenario.
- `example-mcptt-on-network-two-simultaneous-calls.cc` demonstrates two concurrent calls in the same LTE network, each with different MCPTT server nodes, and an MCPTT user participating in both calls
- `mcptt-operational-modes-static.cc` demonstrates how different modes of MCPTT (on-network, off-network, and on-network using UE-to-network relay) operate in parallel, with some support for plotting access time and mouth-to-ear latency KPIs.
- `mcptt-operational-modes-mobility.cc` is similar in initial configuration to the static program above, but demonstrates mobility of three teams of four UEs into a building, and the relative performance impact of the three different modes of operation (on-network, off-network, and on-network with UE-to-network relay).

### mcptt-lte-sl-out-of-covrg-comm

The program `mcptt-lte-sl-out-of-covrg-comm.cc` is an adaptation of the LTE Sidelink example `lte-sl-out-of-covrg-comm.cc` documented in the LTE module. The example was adapted to replace the simple UDP application with MCPTT. The following code excerpts highlight the main aspects of configuring MCPTT in a program.

The first block of code below highlights the use of the class `ns3::psc::McpttHelper` to encapsulate configuration statements on the key objects involved in the MCPTT service. The helper exposes some methods that allow for custom configuration of the `PttApp` class and attributes, `MediaSrc` class and attributes, and `Pusher` class and attributes. In the below, the configuration of `ns3::psc::McpttPusher` to Automatic operation means that the pushing and releasing times will be driven by random variables, and additional methods to easily configure these random variables are provided. Following the configuration of these objects, the usual pattern in *ns-3* for using the application helper to install onto the set of `ueNodes`, configure a start time, and configure a stop time are followed.

```

ApplicationContainer clientApps;
McpttHelper mcpttHelper;
if (enableNsLogs)
{
    mcpttHelper.EnableLogComponents();
}
mcpttHelper.SetPttApp("ns3::psc::McpttPttApp",
    "PeerAddress",
    Ipv4AddressValue(peerAddress),
    "PushOnStart",
    BooleanValue(true));
mcpttHelper.SetMediaSrc("ns3::psc::McpttMediaSrc",
    "Bytes",
    UIntegerValue(msgSize),
    "DataRate",
    DataRateValue(dataRate));
mcpttHelper.SetPusher("ns3::psc::McpttPusher", "Automatic", BooleanValue(true));
mcpttHelper.SetPusherPttInterarrivalTimeVariable("ns3::NormalRandomVariable",
    "Mean",
    DoubleValue(pushTimeMean),
    "Variance",
    DoubleValue(pushTimeVariance));
mcpttHelper.SetPusherPttDurationVariable("ns3::NormalRandomVariable",
    "Mean",
    DoubleValue(releaseTimeMean),
    "Variance",
    DoubleValue(releaseTimeVariance));

clientApps.Add(mcpttHelper.Install(ueNodes));
clientApps.Start(startTime);
clientApps.Stop(stopTime);

```

The above will prepare each UE for the service, but call configuration remains to be configured. For a basic group call type, using the basic floor machine, the helper provides a single statement to configure the call, as follows.

One could also use the following snippet to configure an `ns3::psc::McpttPusherOrchestrator` to control when push and release events occur for a set of `ns3::psc::McpttPttApp` applications.

```

ApplicationContainer clientApps;
McpttHelper mcpttHelper;
if (enableNsLogs)
{
    mcpttHelper.EnableLogComponents();
}
mcpttHelper.SetPttApp("ns3::psc::McpttPttApp",
    "PeerAddress",
    Ipv4AddressValue(peerAddress),
    "PushOnStart",
    BooleanValue(true));
mcpttHelper.SetMediaSrc("ns3::psc::McpttMediaSrc",
    "Bytes",
    UIntegerValue(msgSize),
    "DataRate",
    DataRateValue(dataRate));
mcpttHelper.SetPusher("ns3::psc::McpttPusher", "Automatic", BooleanValue(false));

clientApps.Add(mcpttHelper.Install(ueNodes));

```

(continues on next page)

(continued from previous page)

```

clientApps.Start(startTime);
clientApps.Stop(stopTime);

Ptr<McpttPusherOrchestratorSpurtCdf> orchestrator =
    CreateObject<McpttPusherOrchestratorSpurtCdf>();
orchestrator->SetAttribute("ActivityFactor", DoubleValue(0.5));
orchestrator->StartAt(startTime);

mcpttHelper.Associate(orchestrator, clientApps);

```

The following method:

```
mcpttHelper.ConfigureBasicGrpCall(clientApps, usersPerGroup);
```

encapsulates the following operations:

- sets the call control state machine type to `ns3::psc::McpttCallMachineGrpBasic`
- sets the floor control state machine type to `ns3::psc::McpttOffNetworkFloorParticipant`
- iterates across the `clientApps` in the provided application container. If the provided `usersPerGroup` value is equal to or greater than the size of the `clientApps` container, all instances of `McpttPttApp` will be included in a call with the same `GroupId`. If `usersPerGroup` is less than the size of `clientApps`, the first `usersPerGroup` will be placed into a call with the first `GroupId`, the second `usersPerGroup` will be placed into a call with the second `GroupId`, and so on until the `clientApps` have all been handled. The base `GroupId` is an optional argument to this method, but defaults to the value of 1 if not provided.
- creates the call instance on each `McpttPttApp`

The various configuration variables used in the above are set near the top of the main program, as follows.

```

// MCPTT configuration
uint32_t usersPerGroup = 2;
DataRate dataRate = DataRate("24kb/s");
uint32_t msgSize = 60;           // 60 + RTP header = 60 + 12 = 72
double pushTimeMean = 5.0;      // seconds
double pushTimeVariance = 2.0;  // seconds
double releaseTimeMean = 5.0;   // seconds
double releaseTimeVariance = 2.0; // seconds
Ipv4Address peerAddress = Ipv4Address("225.0.0.0");
Time startTime = Seconds(2);
Time stopTime = simTime;

```

Here, it is worth noting that the configuration sets two users per group, and the example only has two UEs, so both UEs will belong to the same `GroupId`. Also, the `peerAddress` value is set to an IPv4 multicast address. This value should not be changed without similarly changing the IPv4 group address of the TFT configuration further down in the LTE configuration part of the program. The TFT controls the mapping of IP packets to sidelink bearers. If the packets are sent to a peer address for which a suitable TFT is not configured, they will be dropped in the sending UE's stack.

Another difference with respect to the D2D example is that, presently, only IPv4 addressing is supported, so there is no command-line option to use IPv6.

A further difference with respect to the D2D example is that the command-line option to enable `ns-3` logging, `--enableNsLogs`, will not enable LTE logging as in the LTE D2D example, but will instead enable all MCPTT logs, as shown above.

Finally, we note four tracing statements inserted near the bottom of the program:

```
NS_LOG_INFO ("Enabling MCPTT traces...");
mcpttHelper.EnableMsgTraces();
mcpttHelper.EnableStateMachineTraces();
mcpttHelper.EnableMouthToEarLatencyTrace("mcptt-m2e-latency.txt");
mcpttHelper.EnableAccessTimeTrace("mcptt-access-time.txt");
```

These statements are explained in the next section. Some other aspects of LTE tracing are omitted in this modified example, in order to focus on the MCPTT configuration.

### mcptt-on-network-two-calls

The program `mcptt-on-network-two-calls.cc` is an adaptation of the LTE example `lena-simple-epc.cc` to experiment with two MCPTT on-network calls involving two different pairs of UEs to a single MCPTT server managing both calls. The first group call (between the first pair of UEs) starts when the `McpttPttApps` start at time 2 seconds and runs until simulation time 16 seconds. The second call starts at time 18 seconds and runs until time 34 seconds.

This example provides examples of typical statements necessary to configure on-network calls. The first necessary configuration is to add the MCPTT server to the EPC (via the notional IMS) with an `ImsHelper`.

```
Ptr<ImsHelper> imsHelper = CreateObject<ImsHelper>();
imsHelper->ConnectPgw(epcHelper->GetPgwNode());
```

MCPTT clients are added in a manner similar to off-network configuration; e.g.:

```
McpttHelper mcpttClientHelper;
mcpttClientHelper.SetPttApp("ns3::psc::McpttPttApp");
...
clientAppContainer1.Add(mcpttClientHelper.Install(ueNodePair1));
clientAppContainer1.Start(start);
clientAppContainer1.Stop(stop);
```

However, in addition, an MCPTT server app must be configured:

```
Ptr<McpttServerApp> serverApp = DynamicCast<McpttServerApp>(serverAppContainer.
    ↳Get(0));
Ipv4Address serverAddress = Ipv4Address::ConvertFrom(imsHelper->GetImsGmAddress());
serverApp->SetLocalAddress(serverAddress);
```

and in a similar manner, the local IP address of the UEs is configured:

```
for (uint32_t index = 0; index < ueIpIface.GetN(); index++)
{
    Ptr<McpttPttApp> pttApp = clientAppContainer.Get(index)->GetObject<McpttPttApp>();
    Ipv4Address clientAddress = ueIpIface.GetAddress(index);
    pttApp->SetLocalAddress(clientAddress);
    NS_LOG_INFO("client " << index << " ip address = " << clientAddress);
}
```

The next few statements show how to use a `McpttCallHelper` to configure the on-network components of floor arbitrator, the ‘towards participant’ state machine (the component of the floor control at the server that maintains state for each participant), the participant state machine, and properties of the call configurations on the server.

```
McpttCallHelper callHelper;
// Optional statements to tailor the configurable attributes
callHelper.SetArbitrator("ns3::psc::McpttOnNetworkFloorArbitrator",
```

(continues on next page)

(continued from previous page)

```

        "AckRequired", BooleanValue(false),
        "AudioCutIn", BooleanValue(false),
        "DualFloorSupported", BooleanValue(false),
        "TxSsrc", UIntegerValue(100),
        "QueueingSupported", BooleanValue(true));
callHelper.SetTowardsParticipant("ns3::psc::McpttOnNetworkFloorTowardsParticipant",
                                "ReceiveOnly", BooleanValue(false));
callHelper.SetParticipant("ns3::psc::McpttOnNetworkFloorParticipant",
                           "AckRequired", BooleanValue(false),
                           "GenMedia", BooleanValue(true));
callHelper.SetServerCall("ns3::psc::McpttServerCall",
                          "AmbientListening", BooleanValue(false),
                          "TemporaryGroup", BooleanValue(false));

```

The next few statements configure the calls themselves, using the call helper. The user must specify the call type and group ID, the client (McpttPttAtt) applications to configure, the server application, and then the start and stop times of the call.

```

McpttCallMsgFieldCallType callType = McpttCallMsgFieldCallType::BASIC_GROUP;
// Add first call, to start at time 2 and stop at time 10
// Call will involve two nodes (7 and 8) and the MCPTT server (node 3)
uint32_t groupId = 1;
callHelper.AddCall(clientAppContainer1, serverApp, groupId, callType, Seconds(2),
                  ↪Seconds(16));

// Add second call, on new groupId, to start at time 8 and stop at time 15
// Call will involve two nodes (9 and 10) and the MCPTT server (node 3)
groupId = 2;
callHelper.AddCall(clientAppContainer2, serverApp, groupId, callType, Seconds(8),
                  ↪Seconds(34));

```

Finally, the MCPTT tracing can be enabled to trace messages, state machine transitions, and statistics such as mouth-to-ear latency and access time.

```

NS_LOG_INFO("Enabling MCPTT traces...");
McpttTraceHelper traceHelper;
traceHelper.EnableMsgTraces();
traceHelper.EnableStateMachineTraces();
traceHelper.EnableMouthToEarLatencyTrace("mcptt-m2e-latency.txt");
traceHelper.EnableAccessTimeTrace("mcptt-access-time.txt");

```

### mcptt-on-network-two-simultaneous-calls

The program `mcptt-on-network-two-simultaneous-calls.cc` differs from the previously introduced `mcptt-on-network-two-calls.cc` example as follows. First, two calls between three UEs are configured to run concurrently, rather than sequentially, for a duration of nearly 60 seconds. Second, each call has a different MCPTT server. Third, one of the MCPTT users is configured to participate in both calls, and to switch between the two calls every second.

The main change in configuration is to configure a second IMS network (essentially, a second MCPTT server) and to configure its network address to another subnet instead of the default 15.0.0.0:

```

Ptr<ImsHelper> imsHelper2 = CreateObject<ImsHelper>();
imsHelper2->SetImsIpv4Network(Ipv4Address("16.0.0.0"), Ipv4Mask("255.0.0.0"));
imsHelper2->ConnectPgw(epcHelper->GetPgwNode());

```

The first user is added to both the first and second call groups:

```
ApplicationContainer callContainer1;
callContainer1.Add(clientAppContainer1.Get(0));
...
ApplicationContainer callContainer2;
...
// Add the first user to the second call also
callContainer2.Add(clientAppContainer1.Get(0));
```

and the two calls configured in the call helper have different server apps:

```
McpttCallMsgFieldCallType callType = McpttCallMsgFieldCallType::BASIC_GROUP;
uint32_t groupId = 1;
callHelper.AddCall(callContainer1, serverApp, groupId, callType, callStartTime,
↳callStopTime);

groupId = 2;
callHelper.AddCall(callContainer2, serverApp2, groupId, callType, callStartTime,
↳callStopTime);
```

Finally, some events must be added to cause the first MCPTT user to switch calls at specified times. This is done with the `McpttPttApp::SelectCall` method, such as follows:

```
// schedule events to cause the user (McpttPusher) to switch calls at the
// configured times in the simulation
Simulator::Schedule(callStartTime, &McpttPttApp::SelectCall, pttApp, 1, true);
Simulator::Schedule(firstSwitchTime, &McpttPttApp::SelectCall, pttApp, 2, true);
Simulator::Schedule(secondSwitchTime, &McpttPttApp::SelectCall, pttApp, 1, true);
```

The program output of this program shows application and floor control events traced on `userId 1`, illustrating that as one call is selected (sent to the foreground), the other call machine still evolves in the background.

### mcptt-operational-modes-static

The program `mcptt-operational-modes-static.cc` demonstrates how different modes of MCPTT (on-network, off-network, and on-network using UE-to-network relay) operate in parallel, with some support for plotting access time and mouth-to-ear latency KPIs from the resulting traces.

The program creates three teams of UEs (default of 4 UEs per team) with one team in each of the three modes. The off-network and on-network using relay teams are positioned within a building, preventing full on-network operation for those teams. Each team creates an MCPTT call and uses the automatic pusher model to drive an MCPTT call for the duration of the call (default of 1000 seconds duration).

A number of traces are generated:

- `mcptt-operational-modes-static-access-time.dat`: Floor access time trace
- `mcptt-operational-modes-static-m2e-latency.dat`: Talk-spurt mouth-to-ear latency tracing
- `mcptt-operational-modes-static-msg-stats.dat`: MCPTT message trace
- `mcptt-operational-modes-static-pc5-signaling.dat`: Relay PC5 interface trace
- `mcptt-operational-modes-static-positions.dat`: UE position trace
- `mcptt-operational-modes-static-rsrp-measurement.dat`: LTE RSRP measurement trace
- `mcptt-operational-modes-static-state-machine-stats.dat`: MCPTT state machine trace

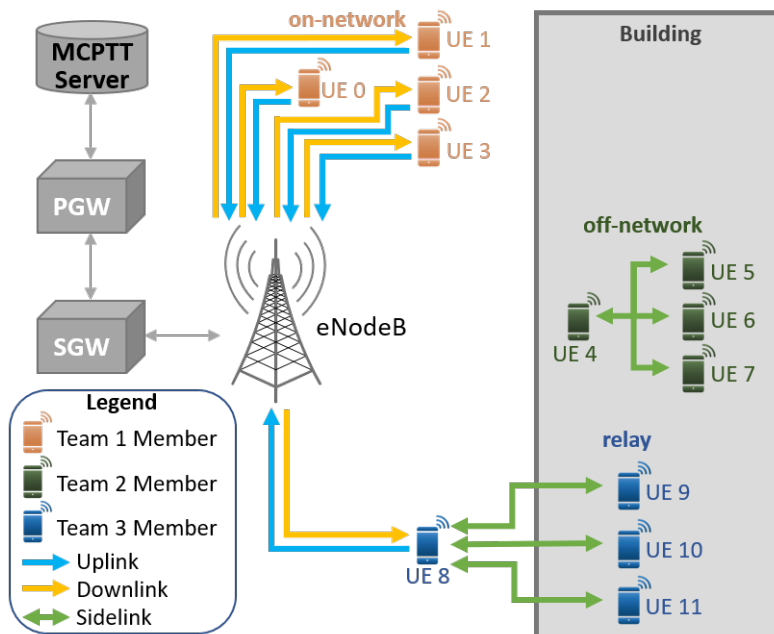


Fig. 4: MCPTT operational modes scenario

At the end of the program execution, some output is printed on the terminal regarding the number of packets sent and received by each node.

The call duration, whether floor control queueing is enabled or not (disabled by default), and the number of UEs per team are configurable as command-line options. A simulation progress display is also optionally enabled by the `--showProgress` option.

Finally, some plotting programs (requiring Python Matplotlib) are available to create CDF plots of the access time and mouth-to-ear latencies of the three teams. The programs are named `mcptt-operational-modes-plot-access-time.py` and `mcptt-operational-modes-plot-m2e-latency.py`. The programs output, by default, an EPS file showing the CDF of access time and mouth-to-ear latency, respectively. They should be run from the same directory as the traces (or the optional program argument can be used to point to the applicable trace file). The following figures show the output produced using the default configuration of `mcptt-operational-modes-static.cc`.

### mcptt-operational-modes-mobility

- `mcptt-operational-modes-mobility.cc` is similar in initial configuration to the static program above, but demonstrates mobility of three teams of four UEs into a building, and the relative performance impact of the three different modes of operation (on-network, off-network, and on-network with UE-to-network relay).

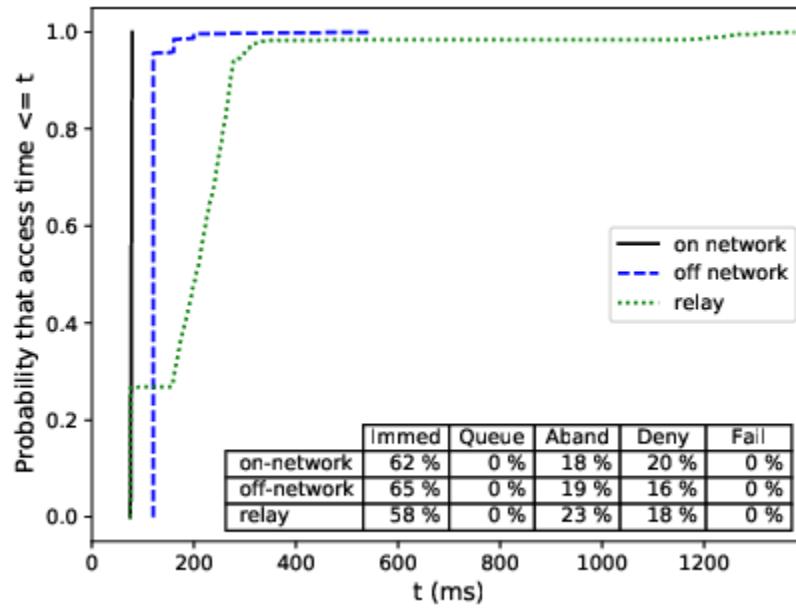


Fig. 5: MCPTT operational modes static access time

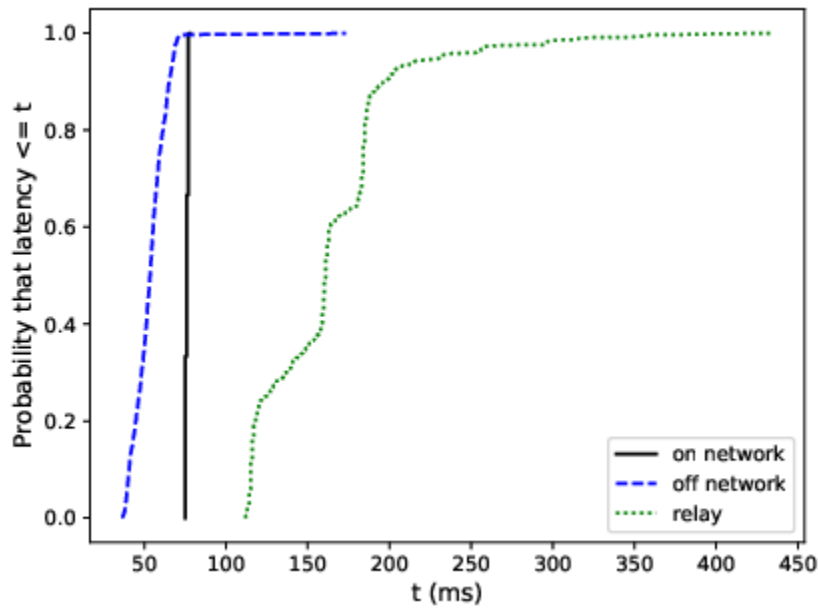


Fig. 6: MCPTT operational modes static mouth-to-ear latency

### 2.3.3 Traces

There are currently four traces that can be activated by using the `ns3::psc::McpttHelper`, and this can be done by following the example given below, after all the applications have been created.

```
NS_LOG_INFO ("Enabling MCPTT traces...");
mcpttHelper.EnableMsgTraces (); // Enable message trace
mcpttHelper.EnableStateMachineTraces (); // Enable state trace
mcpttHelper.EnableMouthToEarLatencyTrace ("mcptt-m2e-latency.txt");
mcpttHelper.EnableAccessTimeTrace ("mcptt-access-time.txt");
```

The `ns3::psc::McpttMsgStats` class is used for tracing MCPTT messages at the application layer and produces a file with the default name “mcptt-msg-stats.txt” with the following file format.

```
time(s) nodeid callid ssrc selected rx/tx bytes message
```

There may also be an additional field at the end of the row called “message” that will be included if the `ns3::psc::McpttMsgStats::IncludeMessageContent` attribute is set to “true”. The “time(s)” column describes the time (in seconds) at which a message was sent/received. The “nodeid” column contains the `ns3::NodeID` value of the sender or receiver, and the “callid” column contains the call ID for this message. The “ssrc” field prints the RTP synchronization source (SSRC) field if available. The “selected” column indicates whether the message was sent or received for the currently selected call on the client, or “N/A” for server messages. The “rx/tx” column indicates if the message was sent or received, i.e., if “rx” is the value in the column then that means the message was received, while a value of “tx” indicates that the message was sent. The “bytes” column indicates the size (in bytes) of the message that was generated at the application layer. The “message” column, if present, includes the string representation of the message that was sent and includes message field names and values.

The `ns3::psc::McpttStateMachineStats` is used for tracing state machine state transitions and produces a file with the default name, “mcptt-state-machine-stats.txt”, with the following format.

```
time(s) userid callid selected typeid oldstate newstate
```

The “time(s)” column describes the time (in seconds) at which the state transition took place. The “userid” column contains the user ID of the MCPTT user that the state machine is associated with. The “callid” column contains the ID of the call the state machine is associated with. The “selected” column indicates whether the message was sent or received for the currently selected call on the client, or “N/A” for server messages. The “typeid” column contains the string representation of the state machine’s `ns3::TypeId`. The “oldstate” column contains the string representation of the `ns3::psc::McpttEntityId` which gives the name of the state that the state machine was in BEFORE the transition took place. And finally, the “newstate” column contains the string representation of the `ns3::psc::McpttEntityId` which gives the name of the state that the state machine was in AFTER the transition took place.

One of the key performance indicators (KPI) defined for MCPTT is the mouth-to-ear latency. More information about this statistic can be found in NIST technical report NISTIR 8206 [NIST.IR.8206]. When the *ns-3* model generates the first RTP packet of a talk spurt, the timestamp is encoded into the payload of that RTP packet and every subsequent RTP packet of the talk spurt. Upon receipt of an `ns3::psc::McpttMediaMsg`, the receiving application will check whether the packet contains a newer ‘start-of-talkspurt’ timestamp. If so, this indicates the reception of the first RTP packet of a new talk spurt from a sender. The latency of all such talk spurts, traced from the perspective of each receiving application, can be traced using the helper method `ns3::psc::McpttHelper::EnableMouthToEarLatencyTrace()`. This method can be called with no arguments so that mouth-to-ear latency samples can be captured in the simulation via the `ns3::psc::McpttHelper::MouthToEarLatencyTrace` traced callback, or a single argument, the trace filename, can be provided so that the collected samples are also written to a file. The output file is formatted as follows.

| time(s)  | ssrc | nodeid | callid | latency(s) |
|----------|------|--------|--------|------------|
| 3.727000 | 3    | 8      | 1      | 0.024010   |
| 3.727000 | 3    | 9      | 1      | 0.024010   |

In the example, node ID 8 started to receive a talk spurt at time 3.727 s, on call ID 1, with a mouth-to-ear latency of 24 ms (see Figure 1 of [NIST.IR.8206]).

The MCPTT Access time is defined as KPI1 in [TS22179], and measures the time between a request to speak (normally by pushing to talk) and when the user receives an indication from floor control to start speaking. In the ns-3 model, this is measured by state transitions of the floor machine; the initial request to speak is marked by a change to a Pending Request state in the MCPTT user's floor control machine, and permission is marked by the transition to the Has Permission state. There can be a few possible state transition outcomes. The first is that the floor control server immediately grants permission. The second is that the request is initially queued but later granted. The third is that the request is denied (perhaps because queuing is disabled or the queue is full). the fourth is that the request ultimately fails, either due to the user (pusher) abandoning the request, or the end of the call, or some other reason.

The latency and outcomes of access requests can be traced using the helper method `ns3::psc::McpttHelper::EnableAccessTimeTrace()`. If no arguments are provided to this method, then samples can be captured in the simulation via the `ns3::psc::McpttHelper::AccessTimeTrace` traced callback, or if a single argument, the trace filename, is provided then the collected samples will also be written to a file. The output file is formatted as follows.

| time(s)   | userid | callid | result | latency(s) |
|-----------|--------|--------|--------|------------|
| 7.246000  | 1      | 0      | I      | 0.024115   |
| 11.251000 | 2      | 0      | Q      | 1.302355   |
| 22.689000 | 4      | 1      | I      | 0.024172   |
| 25.907000 | 3      | 1      | Q      | 2.652518   |
| 28.952000 | 4      | 1      | I      | 0.024378   |
| 34.017000 | 3      | 1      | F      | 0.651460   |

The above example illustrates that at time 7.246 s, user ID 1 from call ID 0 was able to immediately request the floor in a transaction that took a bit more than 24 ms. The timestamp in the first column indicates the time at which the outcome is decided, not when the floor request was initiated. The five possible results described above can be filtered based on the fourth column above, with “I” denoting “immediate”, “Q” denoting “queued”, “D” denoting “denied”, “F” denoting “failed” (such as due to message loss), and “A” denoting “abandoned” (when the PTT button is released while the request is pending).

When measuring access time in ns-3, we recommend to count the “I” and “Q” outcomes and filter out the “D”, “F”, and “A” outcomes. The [TS22179] standard suggests that access time should be less than 300 ms for 99 % of all MCPTT requests, but suggests that the system should have negligible backhaul delay and not be overloaded (less than 70 % load per node) when comparing against this threshold. In simulations, the access time may rise above 300 ms due to non-negligible backhaul delay, the pusher model (if too many MCPTT users are contending for the floor), or congestion or transmission losses in the LTE network.

## 2.4 Testing and Validation

NIST used the 3GPP standards to create over 50 test cases for off-network MCPTT operation. More information about those test cases can be found in NIST technical report NISTIR 8236 [NIST.IR.8236].

On-network MCPTT call control has been tested against the protocol conformance specification 3GPP TS 36.579-2 [TS365792]. The current test suite checks steps 1-11 in Table 6.1.1.3.2-1 for the on-network, on-demand pre-arranged group call with automatic commencement mode. The underlying SIP module supporting the on-network MCPTT call control has also been tested against the SIP RFCs.

On-network MCPTT floor control has been tested for several floor control scenarios, including:

- UE floor release after initiating the call
- UE floor grant after another UE initiates the call and then releases the floor
- UE floor revoked after higher priority UE requests floor
- UE floor denied due to another UE requesting the floor
- UE floor queued and later cancelled by the requesting UE
- UE floor queued and later granted after another UE releases the floor
- Floor control with dual floor control enabled (two UEs holding the floor at the same time)

## UDP GROUP ECHO SERVER

The class `ns3::psc::UdpGroupEchoServer` implements a group echo server that echoes received UDP datagrams to a set of clients. The policy of generating replies can be tailored through the use of attributes.

The relevance of this model to public safety communications is that such scenarios often require a many-to-many group communications application, and existing *ns-3* applications are not suitable to generate such traffic.

The UDP group echo implementation is authored by Fernando J. Cintron ([fernando.cintron@nist.gov](mailto:fernando.cintron@nist.gov)) and is derived from the `UdpEchoServer` found in the *ns-3* applications module.

### 3.1 Model Description

The implementation is provided in the following files:

- `src/psc/model/udp-group-echo-server.{h,cc}` The model itself
- `src/psc/helper/udp-group-echo-helper.{h,cc}` Helper code for configuration

Additionally, a simple example is provided at `src/psc/examples/example-udp-group-echo.cc`.

The model is based on the `ns3::psc::UdpEchoServer`, but differs in that the existing server only handles one client, while the group echo server handles one or more clients. The behavior of the `ns3::psc::UdpEchoServer` can be reproduced (i.e., it is a special case of this object).

The model works as follows. The `UdpGroupEchoServer` is an *ns-3* application, listening for UDP datagrams on a configured UDP port. Upon receipt of a datagram for the first time from a client, the server records the client and a timestamp for when the packet was received. The server then decides whether to forward the packet back to one or more clients, on a pre-configured 'EchoPort'. The set of possible clients is built dynamically based on received packets.

There are a few configurable policies:

1. The server may be configured to echo only to the client that originated the packet (similar to the `UdpEchoServer`).
2. The server may be configured to echo to a group of clients including the sender. Furthermore, the sending client may be excluded from the response.
3. The server may be configured to echo to a group of clients including the sender, so long as the server has heard from each client within a configurable timeout period. Furthermore, the sending client may be excluded from the response.

### 3.1.1 Attributes

The following is the list of attributes:

- **Port:** Port on which the server listens for incoming packets, default value of 9.
- **EchoPort:** Port on which the server echoes packets to client, default value of 0.
- **Mode:** Mode of operation, either no group session (reply to sender only), timeout limited session (replicate to all clients for which a packet has been received from them within the configured timeout period), and infinite session (reply to all known clients). The default is no group session.
- **Timeout:** Inactive client session expiration time, default of zero seconds.
- **EchoClient:** Whether the server echoes back to the sending client, default value of true.,

### 3.1.2 Trace sources

The model also provides a Rx trace source for all received datagrams.

## 3.2 Usage

A simple example based on CSMA links is provided in the file `src/psc/examples/example-udp-group-echo.cc`.

```
// *      *
// n0    n1    ... n(nExtra)  n(1+nExtra)
// |      |      |      |      |
// =====
//      LAN 10.1.2.0
```

By default, node `n0` is the client and node `n1` is the server, although additional nodes can be added with the `--nExtra` argument. If there are more nodes, the highest numbered node is the server.

Each client is configured with an on-off traffic generator that sends traffic at random times to the server. The following program options exercise some of the configuration of the server:

```
$ ./ns3 run 'example-udp-group-echo --PrintHelp'

Program Options:
--nExtra:      Number of "extra" CSMA nodes/devices [0]
--echoClient:  Set EchoClient attribute [true]
--mode:        Set Mode attribute (InfSession|NoGroupSession|TimeoutLimited)
→[InfSession]
--timeout:     Set Timeout attribute [+0.0ns]
--verbose:     Tell echo applications to log if true [true]
--enablePcap:  Enable PCAP file output [false]
--time:        Simulation time [10]
```

By default, the number of extra CSMA nodes/devices is zero, so there will be only one client and one server. By running with the existing defaults, the program will configure the server to echo back to the client, use the 'InfSession' mode of operation (to echo without considering timeout value) and a timeout value of 0 (which is not used in this mode). Logging is also enabled, and running the program with defaults yields output such as:

```
$ ./ns3 run 'example-udp-group-echo'
...

8.7006 server received 41 bytes from 10.1.2.1 port 49153
Client found; old timestamp: 8.6806
New timestamp: 8.7006
8.7006 number of clients: 1
-----
Client      Session
-----
10.1.2.1:49153      0
=====
8.7006 server sent 41 bytes to 10.1.2.1 port 49153
```

This shows that node 10.1.2.1 sent data at time 8.7006 that was echoed back to the client. Running with the option `--nExtra=2` shows:

```
$ ./ns3 run 'example-udp-group-echo --nExtra=2'
...

9.99949 server received 41 bytes from 10.1.2.1 port 49153
Client found; old timestamp: 9.97949
New timestamp: 9.99949
9.99949 number of clients: 3
-----
Client      Session
-----
10.1.2.1:49153      0
10.1.2.2:49153 0.00647334
10.1.2.3:49153 0.00426833
=====
9.99949 server sent 41 bytes to 10.1.2.1 port 49153
9.99949 server sent 41 bytes to 10.1.2.2 port 49153
9.99949 server sent 41 bytes to 10.1.2.3 port 49153
```

Here, one packet sent leads to three packets echoed, and the data under the ‘Session’ column shows the time in seconds since the last packet was received from each client. In mode ‘InfSession’, there is no explicit timeout, but if we set a timeout to be something small, such as 5ms, and set the mode to ‘TimeoutLimited’, we can suppress the response to node 10.1.2.2 because it has last polled the server over 6ms ago.

```
$ ./ns3 run 'example-udp-group-echo --nExtra=2 --timeout=5ms --mode=TimeoutLimited'
...

9.99949 server received 41 bytes from 10.1.2.1 port 49153
9.99949 number of clients: 3
-----
Client      Session
-----
10.1.2.1:49153      0
10.1.2.2:49153 0.00647334 **Session Expired!**
10.1.2.3:49153 0.00426833
=====
9.99949 server sent 41 bytes to 10.1.2.1 port 49153
9.99949 server sent 41 bytes to 10.1.2.3 port 49153
```

As seen above, the session has expired to 10.1.2.2 with this timeout and mode setting, so the echo response is sup-

pressed.

The C++ code for setting the server is fairly standard *ns-3* syntax and container/helper-based code, as exemplified below:

```
uint16_t serverPort = 9;
// Attributes 'timeout', 'mode', 'echoClient' may be set above
UdpGroupEchoServerHelper echoServer (serverPort);
echoServer.SetAttribute ("Timeout", TimeValue (timeout));
echoServer.SetAttribute ("Mode", StringValue (mode));
echoServer.SetAttribute ("EchoClient", BooleanValue (echoClient));
ApplicationContainer serverApps = echoServer.Install (csmaNodes.Get (nCsmas - 1));
serverApps.Start (Seconds (1.0));
serverApps.Stop (Seconds (simTime));
```

## UAV MOBILITY ENERGY MODEL

The class `ns3::psc::UavMobilityEnergyModel` implements an Energy Model that bases the current on how a Node is moving.

This model is roughly based on the energy model released under the GNU GPL v2 found here: [ns3-urbanuavmobility](#)

The model is intended to provide simple energy costs for flying a (single/multi)rotor unmanned aerial vehicle (UAV). The default costs are not intended to perfectly model reality, but rather to consider flight time in cases where otherwise it may have been neglected.

### 4.1 Model Description

The implementation is provided in the following files:

- `src/psc/model/uav-mobility-energy-model.{h,cc}` The model
- `src/psc/model/uav-mobility-energy-model-helper.{h,cc}` A helper

A low-level example may be found in:

`src/psc/examples/uav-mobility-energy-example.cc.`

An example using the helper may be found in:

`src/psc/examples/uav-mobility-energy-helper-example.cc.`

The model sets the current based on five basic modes for movement:

- **STOP:** Not Moving
- **HOVER:** The Node has a velocity of 0 m/s, but the Node is off of the ground ( $z > 0$ )
- **ASCEND/DESCEND:** The Node is moving only on the z-axis
- **MOVE:** The Node is moving on the x-axis or y-axis. Any z movement is added to the calculated current.

The current set when the Node is stopped (not above the ground [ $z = 0$ ]) is zero amps, and is not configurable.

The current set when the Node is hovering is fixed and may be set through the `HoverCurrent` attribute.

The model connects to the `CourseChange` trace in the `MobilityModel` aggregated onto the Node passed to `ConnectMobility()` or `Init()`. When the attached `MobilityModel` notifies this model of a course change this model queries the current velocity and position of the Node. The model will use the velocity and position to determine the state this model should be in (e.g. Ascending, Hovering, Stopped). Each state will incur a different current draw from the attached `EnergySource`. This model then notifies the attached `EnergySource` of this new current via `UpdateEnergySource()`.

When the attached `EnergySource` depletes, this model will be notified and the `EnergyDepleted` trace will be called. However, there is no default behavior beyond this. So, if you want your Node to do something when the attached

EnergySource depletes (i.e. Crash to the ground) then your program must connect to the `EnergyDepleted` trace and implement the desired behavior. See the example for how to go about this.

The current for ascending, descending, and moving is calculated based on the speed at which the Node is moving (except for descending which is based on the inverted speed  $[1/\text{speed}]$ ) multiplied by a conversion factor. Each mode has its own conversion factor and is configurable through the `AscendEnergyConversionFactor`, `DescendEnergyConversionFactor`, and `MoveEnergyConversionFactor` attributes respectively.

### 4.1.1 Attributes

The model exposes the following attributes:

- **AscendEnergyConversionFactor:** The conversion factor applied to the speed of the Node when it is ascending. In  $\frac{A}{m/s}$ .
- **DescendEnergyConversionFactor:** The conversion factor applied to the inverted speed of the Node when it is descending. In  $\frac{A}{(m/s)^{-1}}$ .
- **MoveEnergyConversionFactor:** The conversion factor applied to the speed of the Node when it is moving. In  $\frac{A}{m/s}$ .
- **HoverCurrent:** The fixed amperage set when the Node is hovering. In amperes.
- **EnergySource:** The energy source this model draws from.

### 4.1.2 Trace sources

The model exposes the following trace sources:

- **EnergyDepleted:** Called when the attached Energy Source has indicated that it is depleted.
- **EnergyRecharged:** Called when the attached Energy Source has indicated that it has been recharged.
- **TotalEnergyConsumption:** The total energy consumed by the model. In Joules.
- **State:** The current mobility state detected by the model along with the velocity. The possible states may be found in `ns3::psc::UavMobilityEnergyModel::State`
- **Current:** The current draw from the attached energy source, along with the velocity.

## 4.2 Tests

The following tests have been written for this model and may be found in `src/psc/test/uav-mobility-energy-model-test.cc`:

- **UavMobilityEnergyModelTestCaseEnergyConsumption:** Tests if the model correctly sets the current on the EnergySource as well as if the correct amount of energy is deducted from the EnergySource.
- **UavMobilityEnergyModelTestCaseZeroEnergyCost:** The model supports making any of the possible states consume no energy. Configure each state as such and make sure no energy is consumed.
- **UavMobilityEnergyModelTestCaseMoveEnergy:** Move combines the cost of the 2D movement (x, y) and the ascend/descend cost (z). Tests that when a Node has both 2D movement and is ascending that the ascend/descend cost is accounted for and is correct, along with the 2D movement only.
- **UavMobilityEnergyModelTestCaseLowEnergyTrace:** Tests if the `EnergyDepleted` trace is actually called.

- **UavMobilityEnergyModelTestCaseLiIonEnergySource:** Make sure the model actually draws energy from another EnergySource that is not the BasicEnergySource.
- **UavMobilityEnergyModelTestCaseMobilityModel:** Tests that the model correctly integrates with a MobilityModel. Tests that the MOVE state is correctly determined from the attached MobilityModel
- **UavMobilityEnergyModelTestCaseChangeCostsRuntime:** The model supports changing the cost of any of the states (except for STOP) while the simulation is running. Tests if this feature actually works.
- **UavMobilityEnergyModelTestCaseTotalEnergyConsumption:** The model tracks the total Joules consumed over the lifetime of the model. This total is available through both the `GetTotalEnergyConsumption ()` and through the `TotalEnergyConsumption` trace source. Validate this total is correct from both sources.
- **UavMobilityEnergyModelTestCaseInitialVelocity:** Tests if a velocity set before the simulation starts works correctly.
- **UavMobilityEnergyModelTestCaseState:** Tests if the model correctly sets the state and that the `State` trace has the correct state.
- **UavMobilityEnergyModelTestCaseTraceCurrent;** Tests if the model correctly sets the current and that the `Current` trace has the correct current.
- **UavMobilityEnergyModelTestCaseAttributeEnergySource:** Tests if setting the EnergySource through the `EnergySource` attribute works properly.
- **UavMobilityEnergyModelTestCaseFixedStateChangeSpeed:** Assures that if the speed/velocity at which the Node is moving is changed, but the state does not (i.e. moving, then moving faster in the same direction) that the current is updated (and the `Current` trace notified) and the state remains the same (and the `State` trace is not notified)
- **UavMobilityEnergyModelTestCaseVelocityTraces:** Several of the traces have the velocity from the MobilityModel included. Validate that the included velocities are correct.

## 4.3 Usage

The model may be installed on any Node with a Mobility Model installed. The helper may also be used for a simpler installation. The helper will create any of the necessary components on the Node and install and init the model.

Each Node requires exactly one Energy Source passed to `SetEnergySource ()` and an aggregated MobilityModel when it is passed to `ConnectMobility ()` when using the low-level API. Alternatively, both the EnergySource and Node may be passed to `Init ()` for convenience. If the helper is used, it allows for a passed Energy Source, or an Energy Source aggregated onto the Node.

## INTEL HTTP MODEL

The classes `ns3::psc::IntelHttpClient`, `ns3::psc::IntelHttpHeader`, and `ns3::psc::IntelHttpServer` implement a configurable HTTP model.

This model is an implementation of the model defined in this Intel paper: [A NEW TRAFFIC MODEL FOR CURRENT USER WEB BROWSING BEHAVIOR](#)

### 5.1 Model Description

The implementation is provided in the following files:

- `src/psc/model/intel-http-client.{h,cc}` The client application
- `src/psc/model/intel-http-header.{h,cc}` The header used in packets
- `src/psc/model/intel-http-server.{h,cc}` The server application
- `src/psc/helper/intel-http-helper.{h,cc}` The helper

An example may be found in: `src/psc/examples/example-intel-http.cc`.

This model represents a configurable HTTP client and corresponding HTTP server. The model is based on analysis of logs from ten HTTP Squid proxy caches in the 2007 timeframe (HTTP/1.0 and HTTP/1.1 clients). The log data included requests from 60,000 clients.

In the following description, names containing an ‘Rvs’ suffix denote names of an `ns3::RandomVariableStream` object. Unless otherwise specified, sizes are in Bytes.

The HTTP client generates a request size from `RequestSizeRvs` and sends a request to the server defined by `RemoteAddress` and `RemotePort`.

The server receives the request and generates a size for the response page from `HtmlSizeRvs` as well as the number of embedded objects referenced on generated page. The ‘page’ is then transferred back to the client.

Once the client receives the page, it will wait a time defined by `ParseTimeRvs` to ‘parse’ the received page. Once parsed, the model goes over each embedded object and checks against the probability that it is cached defined by `CacheThreshold`.

If the embedded object is not cached the client will generate another request size (from `RequestSizeRvs`). The server generates Inter-Arrival Time for the embedded object from `EmbeddedObjectIatRvs` and waits that time, generates a size for the object from `EmbeddedObjectSizeRvs` and sends that response. The client then parses that response.

Once all embedded objects have been received, the client will read the completed page for an amount of time defined by `ReadTimeRvs`.

All packets carry an application header, the `IntelHttpRequestHeader`. This header has 3 fields. First, a `Type` field describing whether the packet refers to an HTML object (the main object) or an embedded object. Next, a numeric field (`NumberOfObjects`) with variable meaning, depending on the type of the packet:

- In an HTML Object Request, the field is meaningless.
- In an HTML Object Response, the field details how many embedded objects are associated with the HTML Object served.
- In an Embedded Object Request, the field indicates the sequence number of the embedded object requested within the list of embedded objects for the last HTML object requested.
- In an Embedded Object Response, the field indicates the sequence number of the embedded object served within the list of embedded objects for the last HTML object requested.

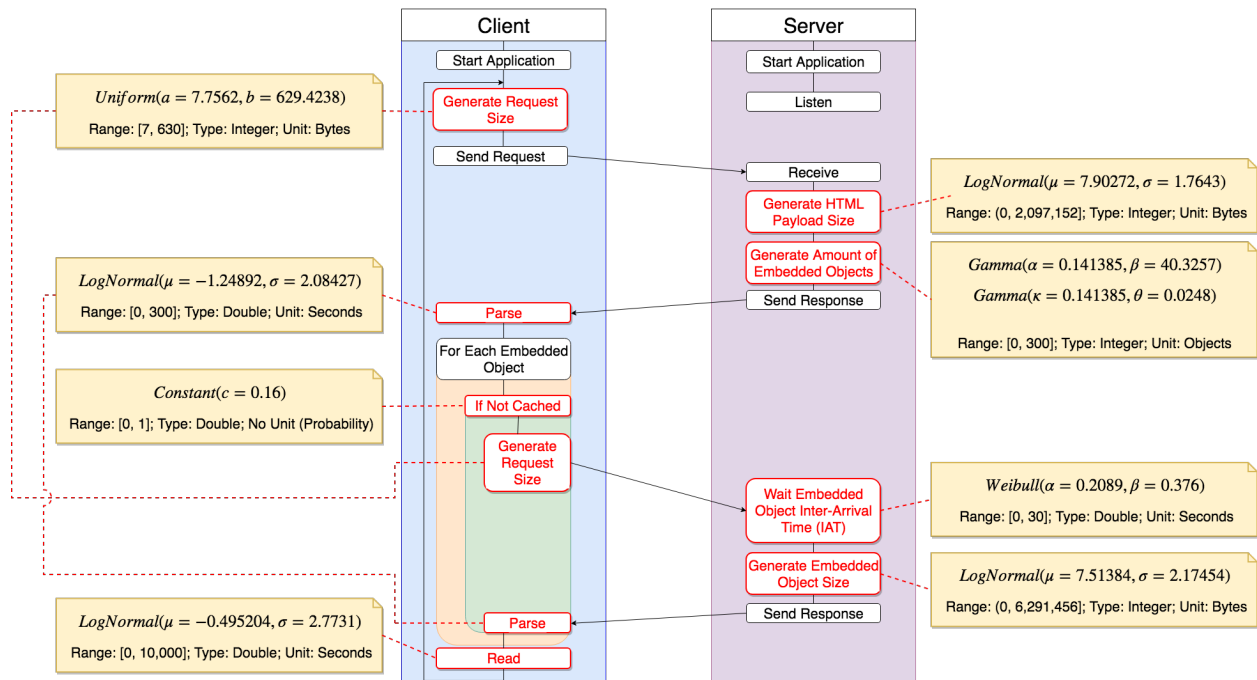
Finally, a `Payload Size` field, with the size of the Request message (for Requests) or the size of the object served (for Responses).

### 5.1.1 Limitations

The `ns3::psc::IntelHttpServer` may only serve one `ns3::psc::IntelHttpClient` at a time.

This restriction may be circumvented by installing several `ns3::psc::IntelHttpServer` applications on one Node on different ports. See the `MultipleServerClients` example in `src/psc/examples/example-intel-http.cc` for a demonstration of one server Node serving many client nodes.

### 5.1.2 Model Flow Diagram



### 5.1.3 Attributes

#### IntelHttpClient

- **RemoteAddress:** The address of the server to communicate with.
- **RemotePort:** The port on the server to send to.
- **CacheThreshold:** Threshold to decide if an embedded object is cached or not. If the value generated by `CacheRvs` is less than or equal to this value, then the object was cached and will not be requested.
- **CacheRvs:** Stream that generates the value compared with the `CacheThreshold` to decide if an embedded object is cached or not. Should generate a value between 0 and 1.
- **RequestSizeRvs:** Random Variable Stream to generate the size of the requests.
- **ParseTimeRvs:** Random Variable Stream to generate the time required to parse a downloaded file. Bound by `ParseTimeLowBound` and `ParseTimeHighBound`.
- **ReadTimeRvs:** Random Variable Stream to generate the time spent reading a page after it has been downloaded. Bound by `ReadTimeLowBound` and `ReadTimeHighBound`.
- **SocketBufferSize:** Size in Bytes of the socket send and receive buffers.

#### IntelHttpHeader

`IntelHttpHeader` exposes no attributes.

#### IntelHttpServer

- **Port:** Port on which we listen for incoming packets.
- **HtmlSizeRvs:** Random Variable Stream to generate the size of the HTML pages. Bound by `HtmlSizeLowBound` and `HtmlSizeHighBound`.
- **EmbeddedObjectAmountRvs:** Random Variable Stream to generate the amount of embedded objects. Bound by `EmbeddedObjectAmountLowBound` and `EmbeddedObjectAmountHighBound`.
- **EmbeddedObjectIatRvs:** Random Variable Stream to generate the IAT of embedded objects. Bound by `EmbeddedObjectIatLowBound` and `EmbeddedObjectIatHighBound`.
- **EmbeddedObjectSizeRvs:** Random Variable Stream to generate the size of the embedded objects. Bound by `EmbeddedObjectSizeLowBound` and `EmbeddedObjectSizeHighBound`.

### 5.1.4 Trace sources

#### IntelHttpClient

- **Rx:** General trace for receiving an application packet of any kind. If the application packet was fragmented, only one line will be traced when the last fragment is received. This trace source is invoked for all application packets received (i.e., all the Responses). This source makes available the received application packet (with the `IntelHttpHeader`) and the address of the node that sent the packet. As the `IntelHttpHeader` is included, trace sinks can make use of the `RequestType` and `NumberOfObjects` fields to match packet entries with the corresponding transmission trace.

- **RxMainObject:** Trace for when an application-level packet containing a main object is fully received. This source is invoked once for each Response with an HTML object (main object). If the Response message was fragmented, the trace will be invoked only when the final fragment has been received. This source makes available the received application packet (with the `IntelHttpRequestHeader`) and the address of the node that sent the packet. As the `IntelHttpRequestHeader` is included, trace sinks can make use of the `RequestType` and `NumberOfObjects` fields to match packet entries with the corresponding transmission trace.
- **RxEmbeddedObject:** Trace for when an application-level packet containing an embedded object is fully received. This source is invoked once for each Response for an embedded object. If the Response message was fragmented, the trace will be invoked only when the final fragment has been received. This source makes available the received application packet (with the `IntelHttpRequestHeader`) and the address of the node that sent the packet. As the `IntelHttpRequestHeader` is included, trace sinks can make use of the `RequestType` and `NumberOfObjects` fields to match packet entries with the corresponding transmission trace.
- **Tx:** General trace for sending an application packet of any kind. This trace source is invoked for all application packets sent (i.e., all the Requests). This source makes available the application packet sent (with the `IntelHttpRequestHeader`). As the `IntelHttpRequestHeader` is included, trace sinks can make use of the `RequestType` and `NumberOfObjects` fields to match packet entries with the corresponding reception trace.
- **TxRequestMainObject:** Trace for when an application-level packet containing a Request for a main object is sent. This source is invoked once for each Request with an HTML object (main object). This source makes available the transmitted application packet (with the `IntelHttpRequestHeader`). As the `IntelHttpRequestHeader` is included, trace sinks can make use of the `RequestType` and `NumberOfObjects` fields to match packet entries with the corresponding reception trace.
- **TxRequestEmbeddedObject:** Trace for when an application-level packet containing a Request for an embedded object is sent. This source is invoked once for each Request with an embedded object. This source makes available the received application packet (with the `IntelHttpRequestHeader`). As the `IntelHttpRequestHeader` is included, trace sinks can make use of the `RequestType` and `NumberOfObjects` fields to match packet entries with the corresponding reception trace.
- **CacheHit:** Trace called when object on a page was cached and does not need to be transmitted.
- **CacheMiss:** Trace called when object on a page was not cached and needs to be requested.

## IntelHttpRequestServer

- **Rx:** General trace for receiving an application packet of any kind. If the application packet was fragmented, only one line will be traced when the last fragment is received. This trace source is invoked for all application packets received (i.e., all the Requests). This source makes available the received application packet (with the `IntelHttpRequestHeader`) and the address of the node that sent the packet. As the `IntelHttpRequestHeader` is included, trace sinks can make use of the `RequestType` and `NumberOfObjects` fields to match packet entries with the corresponding transmission trace.
- **RxRequestMainObject:** Trace for when an application-level packet containing a Request for a main object is fully received. This source is invoked once for each Request with an HTML object (main object). If the request message was fragmented, the trace will be invoked only when the final fragment has been received. This source makes available the received application packet (with the `IntelHttpRequestHeader`) and the address of the node that sent the packet. As the `IntelHttpRequestHeader` is included, trace sinks can make use of the `RequestType` and `NumberOfObjects` fields to match packet entries with the corresponding transmission trace.
- **RxRequestEmbeddedObject:** Trace for when an application-level packet containing a Request for an embedded object is fully received. This source is invoked once for each Request for an embedded object.

If the Request message was fragmented, the trace will be invoked only when the final fragment has been received. This source makes available the received application packet (with the `IntelHttpRequestHeader`) and the address of the node that sent the packet. As the `IntelHttpRequestHeader` is included, trace sinks can make use of the `RequestType` and `NumberOfObjects` fields to match packet entries with the corresponding transmission trace.

- **Tx:** General trace for sending an application packet of any kind. This trace source is invoked for all application packets sent (i.e., all the Responses). This source makes available the application packet sent (with the `IntelHttpRequestHeader`). As the `IntelHttpRequestHeader` is included, trace sinks can make use of the `RequestType` and `NumberOfObjects` fields to match packet entries with the corresponding reception trace.
- **TxMainObject:** Trace for when an application-level packet containing a main object is sent. This source is invoked once for each Response with an HTML object (main object). This source makes available the transmitted application packet (with the `IntelHttpRequestHeader`). As the `IntelHttpRequestHeader` is included, trace sinks can make use of the `RequestType` and `NumberOfObjects` fields to match packet entries with the corresponding reception trace.
- **TxEmbeddedObject:** Trace for when an application-level packet containing an embedded object is sent. This source is invoked once for each Response with an embedded object. This source makes available the received application packet (with the `IntelHttpRequestHeader`). As the `IntelHttpRequestHeader` is included, trace sinks can make use of the `RequestType` and `NumberOfObjects` fields to match packet entries with the corresponding reception trace.

## 5.2 Tests

The following tests have been written for this model and may be found in `src/psc/test/intel-http-model-test.cc`:

- **IntelHttpRequestTestCaseAllCacheHits:** Test with a perfect *CacheThreshold* that no embedded objects are requested/transmitted.
- **IntelHttpRequestTestCaseAllCacheMisses:** Test with a *CacheThreshold* of 0 that all embedded are requested/transmitted.
- **IntelHttpRequestTestCaseCacheTotal:** Verify the total number of cache hits/misses matches the total embedded objects. Also verifies the 'objectsLeft' parameter passed to the *CacheHit* and *CacheMiss* traces.
- **IntelHttpRequestTestCaseEmbeddedObjectSequence:** Tests the, given a constant *EmbeddedObjectLatRvs*, embedded objects are both requested and transmitted in order.
- **IntelHttpRequestTestCaseEventOrder:** Test that each event defined by the *Model Flow Diagram* happens in the correct order.
- **IntelHttpRequestTestCaseHeaderContent:** Verify the request types in each possible packet are of the correct type. As well as the embedded object count.
- **IntelHttpRequestTestCaseIpv6:** Verify the model still works when providing IPv6 addresses instead of IPv4 ones.
- **IntelHttpRequestTestCaseObjectInterArrival:** Verify that, after receiving a request for an embedded object, the server will wait the time from the *EmbeddedObjectLatRvs* before responding.
- **IntelHttpRequestTestCaseParseTime:** Test the the client waits the time from the *RequestSizeRvs* before continuing.
- **IntelHttpRequestTestCaseReadTime:** Test that the client waits the time from *ReadTimeRvs* before requesting another main object.

- **IntelHttpTestCaseTxRxCountSizes:** Verify the specific transmit (i.e., *TxMainObject*) and receive (i.e., *RxEmbeddedObject*) traces are called the same number of times as the general ones (*Tx/Rx*). Also verifies the packets of each type are of the correct size.

## 5.3 Usage

A simple usage example is provided in: `src/psc/examples/example-intel-http.cc`.

The following attributes are exposed for users to change but are set to defaults that match the Intel cache logs, so in practice, most users will not change these values:

- `HtmlSizeRvs`, `HtmlSizeLowBound`, and `HtmlSizeHighBound`.
- **`EmbeddedObjectAmountRvs`, `EmbeddedObjectAmountLowBound`, and `EmbeddedObjectAmountHighBound`.**
- **`EmbeddedObjectIatRvs`, `EmbeddedObjectIatLowBound`, and `EmbeddedObjectIatHighBound`.**
- **`EmbeddedObjectSizeRvs`, `EmbeddedObjectSizeLowBound`, and `EmbeddedObjectSizeHighBound`.**
- `RequestSizeRvs`.
- `ParseTimeRvs`, `ParseTimeLowBound`, and `ParseTimeHighBound`.
- `ReadTimeRvs`, `ReadTimeLowBound`, and `ReadTimeHighBound`.

On the other hand the following attributes are more likely to be changed by users:

- **`RemoteAddress`, `RemotePort`, `Port`:** Besides the basic configuration match between client and server regarding the connection address and port, as the server model only supports a single client, when configuring scenarios with multiple clients special attention must be paid to ensure there are enough servers for all the clients, and that each of them use a unique connection string (address and port).
- **`CacheThreshold`, and `CacheRvs`:** Intel's whitepaper defines the cache mechanism but it does not specify the parameters (threshold or distribution to use). Based on this, the model provides default values for both parameters (`CacheThreshold` and `CacheRvs`) that are functional (a 16 % cache hit ratio on the client fits a small cache size with diverse (i.e., multiple different sites) browsing). However, these values may not fit all use cases (e.g., local caches in corporate environments where browsing is restricted to intranet pages that share the same structure, images, etc. can easily reach 90 % hit ratios).
- **`SocketBufferSize`:** The models configure a socket buffer size set to the maximum size by default (2,147,483,647 Bytes), to avoid the case where the server generates embedded objects with sizes larger than the available socket size. The maximum buffer size allows us to avoid dropping these packets and distorting the distribution configured by the user (or proposed by Intel). However, depending on the area of study and the interests of the users running the simulations, this may lead to unrealistic results, so the parameter is available for configuration as needed (with the caveat that the user will need to ensure that the distribution chosen for the Embedded Object sizes does not generate values that may overflow the buffer).

## VIDEO STREAMING MODEL

The class `ns3::psc::PscVideoStreaming` implements the model of a live video streaming application over UDP. The class `ns3::psc::PscVideoStreamingDistributions` provides the data distributions for some examples of video streaming with specific parameters.

### 6.1 Model Description

The implementation is provided in the following files:

- `src/psc/model/psc-video-streaming.{h,cc}` The video streaming application
- `src/psc/model/psc-video-streaming-distributions.{h,cc}` Data distributions for sample video streams

An example may be found in: `src/psc/examples/example-video-streaming.cc`.

The video streaming application is modeled using statistical approach, in which the traffic generated by the streaming application is characterized by two independent variables that represent the size of the generated packets, and the interval between said packets. These functions are characterized by the Cumulative Distribution Function (CDF) that captures the distribution of values for a given type of video stream. Therefore, a video stream can be defined by just the two CDFs representing the packet size and inter-packet interval. This simple representation makes it easy to extend if needed the data distributions provided as examples.

Different types of video streams will be modeled through different CDFs, so it is important to understand the characteristics of the video stream being modeled in order to assess if any of the provided sample CDFs are a good match for a specific scenario. Some of the aspects that may affect the data distribution are, for example, the video codec used, video resolution, frame rate, brightness of the scene, and amount of movement in the scene.

If new data distributions are needed, the model implementation makes it easy to load the data files from text files, enabling the simulation of videos with characteristics different of those provided by the default distributions.

Additionally, while the model is designed to only work with data distributions of videos with the same characteristics, it is easy to model complex streams with multiple scenes or changing streaming parameters by updating the data distributions used by the model dynamically while a simulation is running. By doing this, it is possible to model complex behaviors by capturing the data distributions of simple video streams, and then combining these simple pieces to build complicated and dynamic behaviors.

The model loads the raw data of the CDFs into two maps: `m_sizeDistribution` for the distribution of the packet size, and `m_intervalDistribution` for the inter-packet interval. If we use one of the provided sample distributions in `src/psc/model/psc-video-streaming-distributions.{h,cc}` we can load the values by using the attribute `Distribution` with one of the following labels:

- “1080p-bright” for H.264 encoded videos at 1920x1080, 60 frames per second, Constant Rate Factor 15, and bright scenes

- “1080p-dark” for H.264 encoded videos at 1920x1080, 60 frames per second, Constant Rate Factor 15, and dark scenes
- “720p-bright” for H.264 encoded videos at 1280x720, 30 frames per second, Constant Rate Factor 25, and bright scenes
- “720p-dark” for H.264 encoded videos at 1280x720, 30 frames per second, Constant Rate Factor 25, and dark scenes

If we want to load our own data distribution, we can do so by calling `ReadCustomDistribution` with the paths to the text files with the CDF for packet size and inter-packet interval as arguments. These files expected to have the following specific format:

- The first line will specify the fixed interval between the probabilities in the CDF.
- The next lines will only contain the values for the probabilities in ascending order, starting at 0 and increasing by the increment read in the first line, up to 1.

The fixed probability increment is a way of ensuring that the CDF size is kept to a reasonable size, as these distributions tend to have long tails that would increase the amount of entries (and therefore, memory and processor cycles used during the simulation) without actual significance for the size or interval.

An example of a custom CDF file for packet size is as follows:

```
0.05
83719
83725
177583
190703
196828
199767
201894
203689
205419
207177
208978
210626
212399
214640
219810
229150
237063
240963
244952
253338
417404
```

This data file corresponds the distribution shown in Figure *Plot of the sample CDF*:

Once the data distributions have been loaded into the maps, the method `LoadCdfs` creates internal random variables of type `ns3::EmpiricalRandomVariable` for each of the distributions. This method is called by the `StartApplication` method, but it can be invoked by other classes to reload the data distributions.

Once the simulation starts, the model will generate a random packet size from the `EmpiricalRandomVariable` (which internally uses a uniform random number to generate a probability, and linear interpolation between the two closest provided values to compute the value associated with the probability generated). If the packet exceeds the size indicated in the attribute `MaxUdpPayloadSize` the packet will be fragmented. Once all the fragments have been sent to the UDP socket, the application waits an interval generated by the inter-packet interval `EmpiricalRandomVariable` before sending another packet.

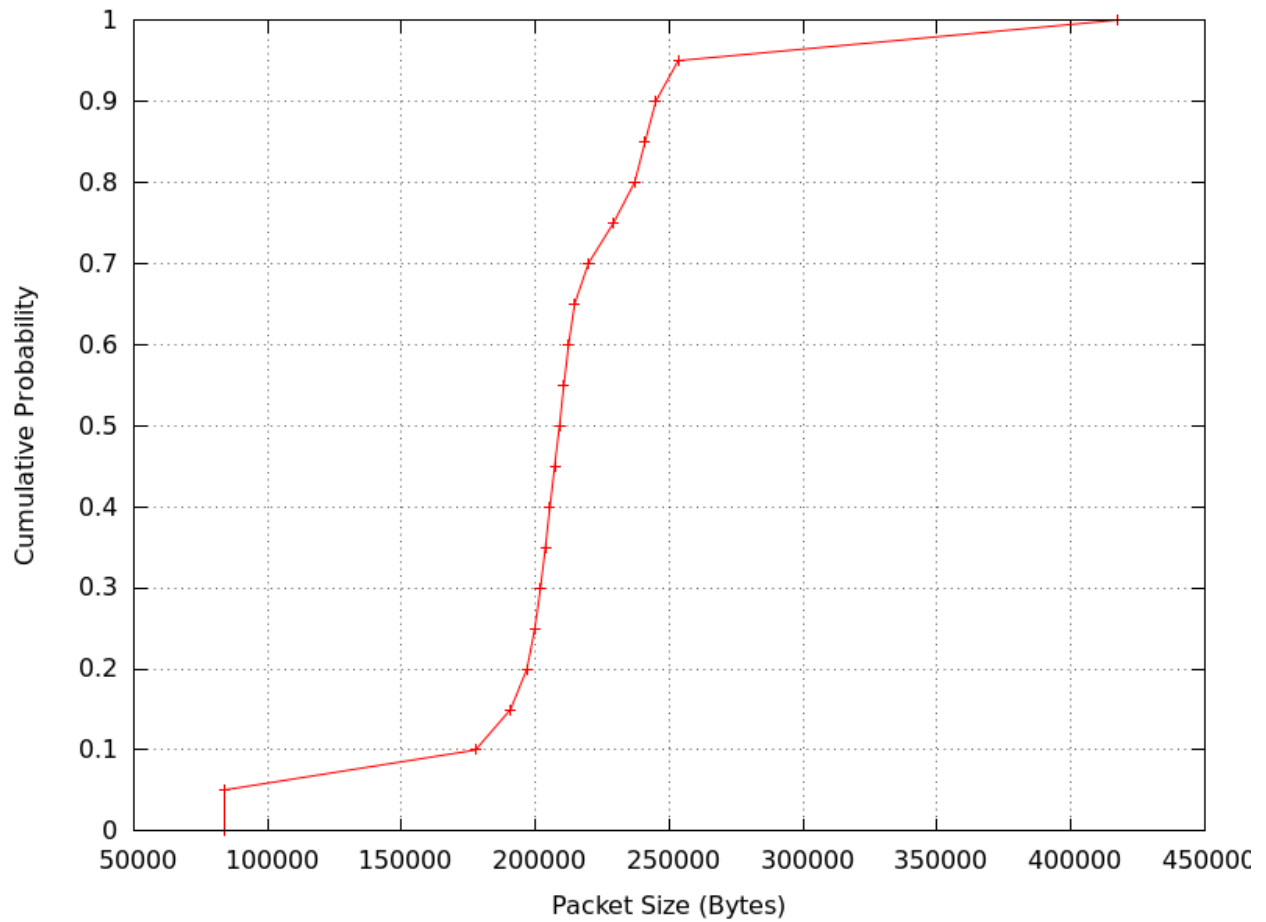


Fig. 1: Plot of the sample CDF

One final feature of the models is that it is possible to configure a period of time at the beginning of the application run (called “Boost Time” in the code) in which the demand of the model is increased. This behavior mimics some streaming applications sending high amounts of data in the first seconds of the stream in order to build a buffer in the client that will provide some protection against unexpected variation of the delay. In the model this is done by only choosing packet sizes of a selected top percentile, and inter-packet intervals of the same bottom percentile (therefore, choosing only between the largest packet sizes with the smallest inter-packet intervals, thus boosting the demand). Once a specified number of packets has been sent with this boosted demand, the complete distributions are used for both packet size and inter-packet interval. To configure this “Boost” period the model provides two attributes:

- **BoostLengthPacketCount** Length of the boosted demand, in packets.
- **BoostPercentile** Top (or bottom) percentile (from 0 to 100) to choose packet sizes and intervals from during the “Boost Time”.

### 6.1.1 Limitations

Currently, custom CDFs must be defined with fixed probability steps. The model can be easily updated to overcome this limitation by interested users.

Changing the data distributions takes effect immediately, which may cause an abrupt change in the application demand, instead of an expected smooth transition.

### 6.1.2 Attributes

#### PscVideoStreaming

- **Distribution:** Label in `ns3::psc::PscVideoStreamingDistributions` that selects the data distribution to be used by the model
- **ReceiverAddress:** Address of the node receiving the data packets
- **ReceiverPort:** UDP port of the application receiving the data packets
- **BoostLengthPacketCount:** Length of the Boost Time expressed as number of packets sent. Set to 0 to indicate no Boost Time is desired.
- **BoostPercentile:** CDF Percentile, between 0 and 100, from which to draw packet sizes and inter-packet intervals during Boost Time. A BoostPercentile of 90 means that for the duration of Boost Time, packet sizes will be selected from the 10 % largest values, while inter-packet intervals will be selected from the 10 % smallest values.
- **MaxUdpPayloadSize:** Maximum payload size for the UDP packets transmitted. If a packet larger than this attribute is generated, it will be fragmented in pieces with, at most, this payload.

### 6.1.3 Trace Sources

#### PscVideoStreaming

- **Tx:** General trace for sending an application packet of any kind. This trace source is invoked for all packets passed to the Socket. Therefore, if a packet is fragmented, this trace source will be invoked once for each fragment.

## 6.2 Usage

A simple usage example is provided in `src/psc/examples/example-video-streaming.cc`.

The data distribution to be used can be loaded from one of the provided distributions through the `Distribution` Attribute:

```
Ptr <PscVideoStreaming> streamingServer = CreateObject<PscVideoStreaming>();
streamingServer->SetAttribute("Distribution", StringValue("1080p-bright"));
```

Alternatively, the distribution data can be loaded from text files (one for the size distribution, one for the inter-packet interval distribution):

```
Ptr <PscVideoStreaming> streamingServer = CreateObject<PscVideoStreaming>();
streamingServer->ReadCustomDistribution(sizeFilePath, intervalFilePath);
```

Once the application starts, the data that was loaded will be used to create some `EmpiricalRandomVariables` for use in the `Send` method. To change the data distribution used (for example, to model the change from a bright scene into a dark scene), we will update the data distribution using one of the methods described above, and then we will invoke the `LoadCdfs` method to signal that the new data should be used:

```
Ptr <PscVideoStreaming> streamingServer = CreateObject<PscVideoStreaming>();
...
streamingServer->ReadCustomDistribution(sizeFilePath, intervalFilePath);
streamingServer->LoadCdfs();
```

## PSC APPLICATION MODEL

The PSC application model provides a simple and flexible way to generate traffic for various use cases using a client/server concept. Specifically, it uses a Request-Response design where the client sends request messages to the server that, if enabled, replies with response messages.

### 7.1 Model Description

The client generates requests following an on-off model, where “on” periods are called sessions. Random variables are associated with the inter-packet interval, number of packets in a session, and the inter-session interval.

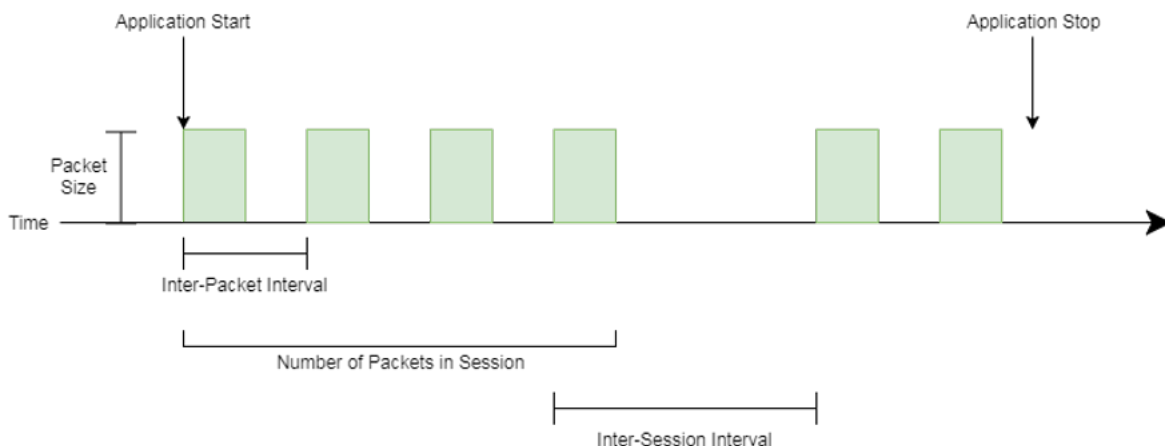


Fig. 1: Client application model

On the server side, a response is generated as soon as a request arrives unless the previous response was sent within the minimum inter-response time, in which case it is delayed. The server application is thus configured using random variables for the packet size and minimum inter-response time.

The implementation is provided in the following files:

- `src/psc/model/psc-application-configuration.{h.cc}` The parameters used by the helper to configure the client and server applications.
- `src/psc/model/psc-application.{h.cc}` Super class for the client and server implementation.
- `src/psc/model/psc-application-client.{h.cc}` The client application sending requests.
- `src/psc/model/psc-application-server.{h.cc}` The server application receiving requests and sending responses.

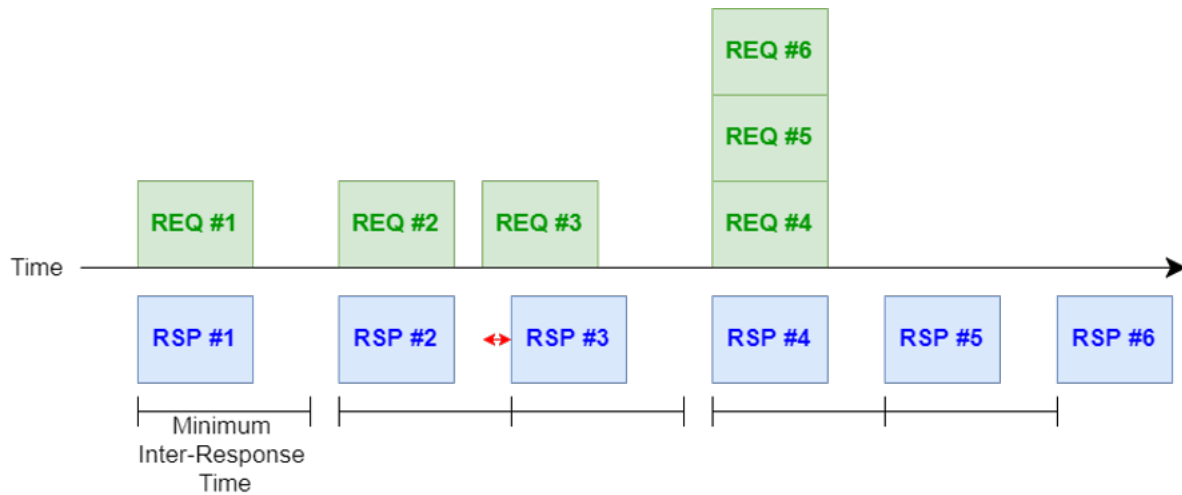


Fig. 2: Server application model

---

**Note:** The application is able to run over any transport protocol, e.g., UDP or TCP.

---

Additionally, a simple example is provided at `src/psc/examples/example-psc-application.cc`.

### 7.1.1 Helpers

One helper, `ns3::psc::PscApplicationHelper`, has been created to facilitate the configuration of the applications in both the client and the server. Currently, the API includes a single function `Install` that takes as parameters the application configure (i.e., name and traffic parameters), the client and server nodes, server address, whether the server should send a response, and the start and stop times of the application. During the install procedure, the helper will configure the name of the application for each node by appending the instance number and the extension “\_Client” or “\_Server” depending on the role of the node. The helper uses an internal variable to the application configuration object to determine the number of times the application has been installed. This is convenient when the same application is deployed for several pairs of nodes. The helper will also configure the random variables in each node associated with the packet sizes, packet intervals, session duration, and time between sessions. The install function will also set the start and stop times of both the client and server, with the server starting 1 s before the client and stopping 1 s after the client in order to limit situations where packets are lost because the server is not ready. The helper returns an `ApplicationContainer` containing first the client then the server.

---

**Note:** The helper currently only works with IPv6 networks.

---

## 7.1.2 Attributes

### PscApplicationConfiguration

PscApplicationConfiguration exposes no attribute.

### PscApplication

- **AppName:** string representing the name of the application.
- **Protocol:** TypeId of the protocol to be used (e.g., UDP).

### PscApplicationClient

- **PacketSize:** Packet size (in bytes) for the requests sent by the client to the server.
- **RemoteAddress:** Address of the server.
- **RemotePort:** Port number of the server application.
- **PacketsPerSession:** Random variable used to select the number of packets to transmit within a session.
- **PacketInterval:** Random variable used to select the interval of time between two packets.
- **SessionInterval:** Random variable used to generate the interval between sessions.

### PscApplicationServer

- **Port:** Listening port to receive packets.
- **PacketSize:** Size of the packets (in bytes) for the responses sent to the client.
- **MinResponseInterval:** Minimum interval between consecutive response packets.
- **IsSink:** Flag to indicate where the server is a sink (i.e., does not send responses to the client) or not.

## 7.1.3 Trace sources

### PscApplication

- **Tx:** General trace for packets transmitted by the application. Each trace includes the name of the application and the SeqTsSizeHeader, allowing tracking of the sequence number, time stamp, and packet size.
- **Rx:** General trace for packets received by the application. Each trace includes the name of the application and the SeqTsSizeHeader, allowing tracking of the sequence number, time stamp, and packet size.
- **Times:** General trace to provide the start and stop times of the application. Each trace includes the name of the application, the start time, and the stop time. The trace is called every time the application is started or stopped.

## 7.2 Usage

There is one example in the `psc/example/example-psc-application.cc`.

In that example, two nodes, `n0` and `n1` are connected via a Carrier Sense Multiple Access (CSMA) link with a 100 Mb/s data rate and 1 ms delay.

```
//
// n0 ===== n1
//   LAN 6001:db80::/64
```

By running with the existing defaults, the program will configure `n0` as the client and `n1` as the server. The client is configured with “on” session durations of 10 s with 5 s between sessions. In each session, the client will send 10 requests of 50 Bytes at 1 s interval. For each request, the server will send a response of 20 Bytes.

The configuration and deployment of the `PscApplication` instances is done in two steps:

- Creation and configuration of a `PscApplicationConfiguration` object as shown below:

```
Ptr<PscApplicationConfiguration> appConfig =
    CreateObject<PscApplicationConfiguration>("PscApplicationExample",
        UdpSocketFactory::GetTypeId(), // Socket type
        5000                          // port number
    );

appConfig->SetApplicationPattern(
    CreateObjectWithAttributes<ConstantRandomVariable>(
        "Constant", DoubleValue(10)), // Number of packets to send per session
    CreateObjectWithAttributes<ConstantRandomVariable>(
        "Constant", DoubleValue(1)), // Packet interval (in s)
    CreateObjectWithAttributes<ConstantRandomVariable>("Constant",
        DoubleValue(5)), // Time between sessions
    50, // Client packet size (bytes)
    20); // Server packet size (bytes)
```

- Deployment of the application in the client and server nodes, which can easily be done using an instance of `PscApplicationHelper` as show below:

```
Ptr<PscApplicationHelper> appHelper = CreateObject<PscApplicationHelper>();

ApplicationContainer apps = appHelper->Install(
    appConfig,
    csmaNodes.Get(0),
    csmaNodes.Get(1),
    csmaNodes.Get(1)->GetObject<Ipv6L3Protocol>()->GetAddress(1, 1).GetAddress(),
    echoClient,
    Seconds(startTime),
    Seconds(simTime));
```

The example program supports a few options to change the application behavior and the level of output. The list of arguments is listed by using the ‘`—PrintHelp`’ option shown below:

```
$ ./ns3 --run 'example-psc-application --PrintHelp'

Program Options:
  --echoClient:    Set EchoClient attribute [true]
  --verbose:       Tell echo applications to log if true [true]
```

(continues on next page)

(continued from previous page)

```
--enableTraces:  Enable trace file output [false]
--time:          Simulation time [30]
```

The “echoClient” argument controls whether the server will send responses to the client. The “verbose” argument enables debug information and requires the program to be compile with ‘debug’ option. The “enableTraces” lets the program connect to the application trace sources and store the packets transmitted and received by both the client and the server applications. Finally, the “time” argument controls the duration of the simulation.

By enabling the traces, the program will generate an output file called PscApplicationExample\_trace.txt with the packet information. If the “echoClient” argument is set to “true” (default), the output will look like:

```
Name      Time      Action  Payload SeqNum
PscApplicationExample_0_Client 2 TX 50 1
PscApplicationExample_0_Server 2.00503 RX 50 1
PscApplicationExample_0_Server 2.00503 TX 20 2
PscApplicationExample_0_Client 2.00604 RX 20 2
PscApplicationExample_0_Client 3 TX 50 3
PscApplicationExample_0_Server 3.00101 RX 50 3
PscApplicationExample_0_Server 3.00101 TX 20 4
PscApplicationExample_0_Client 3.00202 RX 20 4
PscApplicationExample_0_Client 4 TX 50 5
PscApplicationExample_0_Server 4.00101 RX 50 5
...
```

If the “echoClient” argument is set to “false”, the output will look like:

```
Name      Time      Action  Payload SeqNum
PscApplicationExample_0_Client 2 TX 50 1
PscApplicationExample_0_Server 2.00503 RX 50 1
PscApplicationExample_0_Client 3 TX 50 2
PscApplicationExample_0_Server 3.00101 RX 50 2
PscApplicationExample_0_Client 4 TX 50 3
PscApplicationExample_0_Server 4.00101 RX 50 3
PscApplicationExample_0_Client 5 TX 50 4
PscApplicationExample_0_Server 5.00101 RX 50 4
PscApplicationExample_0_Client 6 TX 50 5
PscApplicationExample_0_Server 6.00101 RX 50 5
...
```

## INCIDENT SCENARIO FRAMEWORK

In [NIST2019b], guidelines were provided to help authors of scenarios document incidents with enough detail to be reproducible by different researchers using different platforms. In addition, the guidelines separated the description of the incident and the communication needs (who, what, where) from the technology (how), thus making it easier to investigate different network capabilities for the same scenario.

Similarly, the objective of the incident scenario framework presented in this section is to provide a set of classes and a methodology for developing potentially complex scenarios involving hundreds of users and a multitude of applications. The modular approach helps the users evaluate different technologies and configurations without having to start from scratch.

**Note:** The full scenario is large and runs for over four hours of simulation time. The scenario will trigger an assert or exception if the LTE traces feature is enabled or if asserts are enabled in the build. These are apparently due to upstream LTE issues that have not been resolved as of release v7.0. In addition, LTE traces are verbose and will consume tens of GB of file system if enabled, and will cause slower execution time. Therefore, LTE traces are disabled by default, and users are encouraged to build ns-3 in optimized mode without asserts enabled (the default setting). For example:

```
$ ./ns3 configure -d optimized --enable-examples --enable-tests
```

### 8.1 Design Overview

The methodology for implementing scenarios involves the following steps:

1. Describe the scenario: define the area where the incident occurs, including the buildings if present, all the users and their mobility, and the timing of the main events.
2. Configure the network: configure technology specific devices in the nodes. For example, deploy an LTE network and attach NetDevices to the nodes.
3. Configure the applications: deploy applications in the nodes with appropriate start and stop time based on the scenario events.

This methodology was used to implement the fictional school shooting scenario described in [NIST2021c] and whose implementation is located in `psc/example/schoolshooting`.

To assist with the description of scenarios and provide consistent information, a new class `ns3::psc::PscScenarioDefinition` was created to define and store scenario information composed of:

- Areas: an area is a zone of interest in the scenario. When an incident occurs, first responders typically define perimeters and areas where specific activities will take place (e.g., triage). Those areas are defined by a name and a rectangle.

- Structures: a structure represents a building at the location of the incident. A structure is defined by a name and a BuildingContainer to store one or more ns-3 buildings.
- Node groups: a group of nodes associated with a given role. Typical response teams are organized by roles, e.g., SWAT teams or firefighters. A group of nodes is defined by the name of the group and a NodeContainer with one or more ns-3 nodes.
- Key events: an event is used to identify the time at which a significant event occurred during the scenario, e.g., an explosion or an arrest. Those events are usually followed by actions from the first responders. An event is defined by a name and the time at which it occurs.
- Applications: a list of applications used during the incident.

The `ns3::psc::PscScenarioDefinition` stores the information using different maps with a user-provided unique ID as the key.

The implementation is provided in the following file:

- `src/psc/model/psc-scenario-definition.{h.cc}`.

### 8.1.1 Helpers

The `ns3::psc::PscScenarioTraceHelper` class defined by the files `src/psc/helper/psc-scenario-definition.{h.cc}` can be used to provide consistent output traces for any incident scenario. The traces are generated at the time they are enabled, therefore the user must define the scenario completely before attaching the helper otherwise some information may not be included in the trace files.

The name of the trace files is based on the scenario name and append a suffix based on the type of trace and the file extension “.txt”.

The trace file for the areas has a name that ends with the suffix “-areas.txt” and has the following format:

|    |      |      |      |      |      |
|----|------|------|------|------|------|
| ID | Name | xMin | xMax | yMin | yMax |
|----|------|------|------|------|------|

where xMin, xMax, yMin, and yMax define the boundaries of the area.

The trace file for the structures has a name that ends with the suffix “-structures.txt” and has the following format:

|    |      |      |      |      |      |      |      |
|----|------|------|------|------|------|------|------|
| ID | Name | xMin | xMax | yMin | yMax | zMin | zMax |
|----|------|------|------|------|------|------|------|

where xMin, xMax, yMin, yMax, zMin, and zMax define the 3D boundaries of the structure (i.e., ns-3 building).

The trace file for the nodes has a name that ends with the suffix “-nodes.txt” and has the following format:

|         |           |     |   |   |   |
|---------|-----------|-----|---|---|---|
| GroupID | GroupName | UID | X | Y | Z |
|---------|-----------|-----|---|---|---|

where UID is ns-3 generated node ID, and X, Y, and Z are the initial coordinates for the node.

The trace file for the events has a name that ends with the suffix “-events.txt” and has the following format:

|    |      |      |
|----|------|------|
| ID | Name | Time |
|----|------|------|

where Time is the time of the event.

Additionally, the helper can be used to trace the progress of the simulation by calling the ‘EnableTimeTrace’ method and providing an `ns3::Time` argument that specifies the time interval between updates. The simulation time is then printed into a file with the suffix “-time.txt”.

### 8.1.2 Attributes

The class `PscScenarioDefinition` does not expose any attribute.

### 8.1.3 Trace sources

The class `PscScenarioDefinition` does not expose any trace source.

## 8.2 Example: School shooting

The example available in the `psc/example/schoolshooting` folder is a large scale school shooting incident involving over 100 first responders and vehicles, more than a dozen different types of applications, and that is over four hours in simulation time.

The implementation of the example is provided in the following files:

- `src/psc/examples/schoolshooting/psc-schoolshooting.cc` The main program to launch the example.
- `src/psc/examples/schoolshooting/schoolshooting-definition-helper.{h.cc}` Helper class to define the scenario.
- `src/psc/examples/schoolshooting/schoolshooting-network-technology-helper.{h.cc}` Abstract helper class that defines the API for the underlying technology helper.
- `src/psc/examples/schoolshooting/schoolshooting-lte-helper.{h.cc}` Helper class for deploying the LTE network supporting the incident that implements the underlying technology API.
- `src/psc/examples/schoolshooting/schoolshooting-application-helper.{h.cc}` Helper class to define and deploy the applications.
- `src/psc/examples/schoolshooting/schoolshooting-application-trace-helper.{h.cc}` Helper class to activate application traces.
- `src/psc/examples/schoolshooting/schoolshooting-lte-vis-helper.{h.cc}` Helper class to visualize the incident using NetSimulyzer [NIST2021b].

### 8.2.1 Helpers

#### **SchoolShootingDefinitionHelper**

The `SchoolShootingDefinitionHelper` helper is responsible for defining the school shooting scenario description. All the areas, buildings, nodes, and events are defined in this class. The helper also defines the mobility models of each node throughout the duration of the incident.

### SchoolShootingNetworkTechnologyHelper

The `SchoolShootingNetworkTechnologyHelper` helper is meant to provide an API for technology specific deployment helpers. It provides virtual functions to access nodes specified in the scenario that may be implemented differently based on the technology used.

### SchoolShootingLteHelper

The `SchoolShootingLteHelper` helper is a subclass of `SchoolShootingNetworkTechnologyHelper` and is responsible for deploying an LTE network and associated devices in order to provide first responders with a means of communication.

### SchoolShootingApplicationHelper

The `SchoolShootingApplicationHelper` helper is the interface towards the applications. It is responsible for deploying all the applications inside the nodes. It manages the ports to be used by each application. Getter functions are also available to access the installed applications during simulation. Finally, the helper API also allows the user to enable traces for the various applications.

### SchoolShootingApplicationTraceHelper

The `SchoolShootingApplicationHelper` helper is responsible for creating trace files associated with the various applications. When an application trace is enabled, the helper will connect to the model's trace sources and register callback functions to write traces to files. Because the scenario uses different types of applications, the helper functions are specific to each application model.

### SchoolShootingLteVisHelper

The `SchoolShootingLteVisHelper` helper is an optional helper that generates a JSON file to replay and visualize the incident with NetSimulyzer [NIST2021b]. It connects to the scenario definition to obtain building and node information (including positions throughout the simulation), to the technology helper to obtain network information (e.g., eNodeB height), and to the application helper to connect to the applications' trace sources.

## 8.2.2 Attributes

### SchoolShootingDefinitionHelper

- `CreateClassrooms`: boolean to enable the creation of ns-3 buildings representing the classrooms inside the main buildings. Due to limitations in the ns-3 propagation models with buildings inside other buildings, it should not be used when a technology is enabled. For example, this feature can be enabled with the scenario is ran with the NetSimulyzer traces enabled and the LTE disabled.

## SchoolShootingNetworkTechnologyHelper

The class `SchoolShootingNetworkTechnologyHelper` does not expose any attribute.

## SchoolShootingLteHelper

- `PathLossModelType`: the path loss model to use. The default is `ns3::Hybrid3gppPropagationLossModel`.
- `SchedulerType`: the MAC scheduler. The default is `ns3::RrFfMacScheduler`.
- `DlBandwidth`: the number of RBs in the downlink. The default is 50 RBs.
- `UlBandwidth`: the number of RBs in the uplink. The default is 50 RBs.
- `DlEarfcn`: the Downlink E-UTRA Absolute Radio Frequency Channel Number (EARFCN) as per 3GPP 36.101. The default is 5330 (LTE Band 14).
- `UlEarfcn`: the Uplink E-UTRA Absolute Radio Frequency Channel Number (EARFCN) as per 3GPP 36.101. The default is 23 330 (LTE Band 14).
- `UeAntennaHeight`: the height of the UEs. The default is 1.5 m.
- `EnbAntennaHeight`: the height of the eNodeBs. The default is 30 m.
- `UeTransmissionPower`: the UE transmit power. The default is 23 dBm.
- `EnbTransmissionPower`: the eNodeB transmit power. The default is 46 dBm.
- `DefaultTransmissionMode`: the eNodeB transmission mode. The default is 2 (MIMO spatial multiplexing).
- `LinkDataRate`: the rate for the links between network elements. The default is 10 Gb/s.
- `LinkDelay`: the delay for the links between network elements. The default is 10 ms.

## SchoolShootingApplicationHelper

- `Testing`: boolean to enable/disable the testing mode of the applications. If enabled, applications are configured with condensed parameters and start/stop times below 100 s.

## SchoolShootingApplicationTraceHelper

The class `SchoolShootingApplicationTraceHelper` does not expose any attribute.

## SchoolShootingLteVisHelper

- `Interval`: time interval between node mobility updates. Smaller values increase the size of the output file.

### 8.2.3 Trace sources

None of the helper classes exposes trace sources. However, the helpers `SchoolShootingApplicationHelper` and `SchoolShootingLteVisHelper` connect to many different trace sources exposed by the technology and application models.

### 8.2.4 Usage

The main program located in `src/psc/examples/schoolshooting/psc-schoolshooting.cc` is responsible for assembling the various components of the simulation. Thanks to the use of helper classes, the code is relatively simple.

The following code block shows the minimum setup to create a simulation based on the school shooting scenario:

```
// 1. Create the scenario
Ptr<SchoolShootingDefinitionHelper> scenarioDefinitionHelper =
    CreateObject<SchoolShootingDefinitionHelper>();

// 2. Create technology helper and initialize the network
Ptr<SchoolShootingLteHelper> lteScenarioHelper = CreateObject<SchoolShootingLteHelper>
    ();
lteScenarioHelper->SetScenarioDefinitionHelper(scenarioDefinitionHelper);
lteScenarioHelper->Initialize();

// 3. Create the application helper and
Ptr<SchoolShootingApplicationHelper> scenarioApplicationHelper =
    CreateObject<SchoolShootingApplicationHelper>();
scenarioApplicationHelper->SetTechnologyHelper(lteScenarioHelper);
scenarioApplicationHelper->SetScenarioDefinitionHelper(scenarioDefinitionHelper);

// 4. Enable traces
Ptr<PscScenarioTraceHelper> scenarioTraceHelper =
    CreateObject<PscScenarioTraceHelper>(scenarioDefinitionHelper->
    GetScenarioDefinition());
scenarioTraceHelper->EnableScenarioTraces();
scenarioApplicationHelper->EnableApplicationTraces();
lteScenarioHelper->EnableLteTraces();
```

However, for convenience, the example program supports a few options to change the application behavior and the trace selection. The list of arguments is listed by using the ‘`--PrintHelp`’ option shown below:

```
$ ./ns3 run 'psc-schoolshooting --PrintHelp'

Program Options:
  --duration:          Duration (in Seconds) of the simulation [+4.0833h]
  --enableLte:         Flag to enable LTE deployment [true]
  --enableLteTraces:   Flag to enable LTE traces [true]
  --enableGuiTraces:   Flag to enable the visualization traces [false]
  --guiResolution:     Granularity of the visualization [1000]
  --reportTime:        Indicates whether or not the simulation time is reported
  --periodically [+1s]
  --testing:           Indicates if using modified application times for 100 s
  --simulation [false]
  --appId:             The ID for the application (see schoolshooting-application-
  --helper.h::SchoolShootingApplicationId [-1])
```

The “duration” argument allows the user to control the duration of the simulation, but there is no traffic after the

incident ends at 14700 s (245 min). The “enableLte” argument determines if the LTE technology is enabled. If disabled, buildings and nodes are still created, but applications cannot run. Disabling it is useful to verify node mobility as the simulation speed drastically improves. If LTE is enabled, application traces are also enabled. The name of the scenario is also based on whether or not LTE is enabled. If LTE is disabled, the scenario name is “SchoolShootingNoLte”, and if it is enabled, the name is “SchoolShootingLte”, thus avoiding overwriting output files. The “enableLteTraces” controls the generation of the standard LTE traces, some of which are very large due to the simulation length. The “enableGuiTraces” and “guiResolution” enables the generation of a trace file for NetSimulyzer and the frequency of updates, respectively. Note that if NetSimulyzer is not part of the ns-3 installation, enabling the traces will be ignored. The “reportTime” argument sets the periodicity of the time trace that prints the current simulation time. This is useful to monitor the progress of the simulation which can be a couple of days depending on the machine on which the simulation is running. The “testing” argument can be used to toggle a modified version of the scenario in which applications parameters are condensed and the simulation is set to 100 s. As the name indicates, this was used for testing the scenario and detecting potential issues without running the full simulation. Finally, the “appId” argument can be used to run the simulation by enabling a single type of application as defined in the enumeration `SchoolShootingApplicationHelper::SchoolShootingApplicationId`. This was used to validate the application configuration. Note that the behavior observed with a single application enabled does not necessarily reflect the behavior that will be observed when all the applications are enabled due to the possibility of congestion and packet loss.

In addition to the parameters exposed as command-line arguments, ns-3 allows the configuration of the default value of any class attribute through the command-line interface, and allows available classes and attributes to be printed out. See the [ns-3 tutorial](#) for more information about this.

A number of output files are generated based on the traces enabled. In addition to scenario files, application-specific traces are also generated. The number and format of the traces is dependent on the model being used. By default the following files will be created:

- Scenario information:
  - `SchoolShootingLte-structures.txt`: list of buildings and their locations.
  - `SchoolShootingLte-areas.txt`: list of areas and their boundaries.
  - `SchoolShootingLte-nodes.txt`: list of nodes including their group name and initial location.
  - `SchoolShootingLte-events.txt`: list of events and the time they occur.
  - `SchoolShootingLte-time.txt`: file tracking the simulation time.
- Application statistics:
  - `SchoolShootingLte-AutomaticVehicleLocation-appTraffic.txt`: trace of packets sent and received by the instances of the vehicle location tracking application. Generated by the `PscApplication` model.
  - `SchoolShootingLte-AutomaticVehicleLocation-appTimes.txt`: trace of start and stop times of the instances of the vehicle location tracking application. Generated by the `PscApplication` model.
  - `SchoolShootingLte-Biometrics-appTraffic.txt`: trace of packets sent and received by the instances of the biometrics application. Generated by the `PscApplication` model.
  - `SchoolShootingLte-Biometrics-appTimes.txt`: trace of start and stop times of the instances of the biometrics application. Generated by the `PscApplication` model.
  - `SchoolShootingLte-BuildingPlan-appTraffic.txt`: trace of packets sent and received by the instances of the building plans data application. Generated by the `PscApplication` model.
  - `SchoolShootingLte-BuildingPlan-appTimes.txt`: trace of start and stop times of the instances of the building plans data application. Generated by the `PscApplication` model.

- SchoolShootingLte-DeployableCameraVideo-appTraffic.txt: trace of packets sent and received by the instances of the deployable camera. Generated by the PscVideoStreaming model.
- SchoolShootingLte-EmsVehicleCameraVideo-appTraffic.txt: trace of packets sent and received by the instances of the EMS vehicle camera. Generated by the PscVideoStreaming model.
- SchoolShootingLte-HelicopterCameraVideo-appTraffic.txt: trace of packets sent and received by the instances of the helicopter camera. Generated by the PscVideoStreaming model.
- SchoolShootingLte-IncidentManagement-appTraffic.txt: trace of packets sent and received by the instances of the incident management application. Generated by the PscApplication model.
- SchoolShootingLte-IncidentManagement-appTimes.txt: trace of start and stop times of the instances of the incident management application. Generated by the PscApplication model.
- SchoolShootingLte-NG911Video-appTraffic.txt: trace of packets sent and received by the instances of the NG911 videos. Generated by the PscVideoStreaming model.
- SchoolShootingLte-RmsBca-appTraffic.txt: trace of packets sent and received by the instances of the Records Management System (RMS) and Bureau of Criminal Apprehension (BCA) database application. Generated by the PscApplication model.
- SchoolShootingLte-RmsBca-appTimes.txt: trace of start and stop times of the instances of the RMS and BCA database application. Generated by the PscApplication model.
- SchoolShootingLte-Satellite-appTraffic.txt: trace of packets sent and received by the instances of the satellite images application. Generated by the PscApplication model.
- SchoolShootingLte-Satellite-appTimes.txt: trace of start and stop times of the instances of the satellite images application. Generated by the PscApplication model.
- SchoolShootingLte-SchoolCameraVideo-appTraffic.txt: trace of packets sent and received by the instances of the school camera. Generated by the PscVideoStreaming model.
- SchoolShootingLte-SwatHelmetCameraVideo-appTraffic.txt: trace of packets sent and received by the instances of the helmet cameras. Generated by the PscVideoStreaming model.
- SchoolShootingLte-TacticalTelemetry-appTraffic.txt: trace of packets sent and received by the instances of the tactical telemetry application. Generated by the PscApplication model.
- SchoolShootingLte-TacticalTelemetry-appTimes.txt: trace of start and stop times of the instances of the tactical telemetry application. Generated by the PscApplication model.
- SchoolShootingLte-ThrowPhone-appTraffic.txt: trace of packets sent and received by the instances of the throw phone voice application. Generated by the PscApplication model.
- SchoolShootingLte-ThrowPhone-appTimes.txt: trace of start and stop times of the instances of the throw phone voice application. Generated by the PscApplication model.
- SchoolShootingLte-ThrowPhoneCameraVideo-appTraffic.txt: trace of packets sent and received by the instances of the throw phone camera. Generated by the PscVideoStreaming model.
- SchoolShootingLte-TrafficCameraVideo-appTraffic.txt: trace of packets sent and received by the instances of the traffic camera. Generated by the PscVideoStreaming model.
- SchoolShootingLte-VideoConference-appTraffic.txt: trace of packets sent and received by the instances of the audio conference application. Generated by the PscApplication model.
- SchoolShootingLte-VideoConference-appTimes.txt: trace of start and stop times of the instances of the audio conference application. Generated by the PscApplication model.

- `SchoolShootingLte-VideoConferenceVideo-appTraffic.txt`: trace of packets sent and received by the instances of the video conference application. Generated by the `PscVideoStreaming` model.
  - `SchoolShootingLte-Web-appTraffic.txt`: trace of packets sent and received by the instances of the web application. Generated by the `IntelHttp` model.
  - `mcptt-m2e-latency.txt`: MCPTT related trace containing mouth-to-ear latency. Generated by the MCPTT application model.
  - `mcptt-access-time.txt`: MCPTT related trace containing access time performance. Generated by the MCPTT application model.
- **LTE traces** Additionally, there are 13 LTE trace files to monitor the uplink and downlink transmissions at the physical layers all the way to the PDCP layers. The format of the files is described in the LTE section of the ns-3 model documentation.

If the NetSimulyzer traces are enabled, the following file is also generated:

- `SchoolShootingLte.json`: file containing visualization information for NetSimulyzer. Once loaded, the user can visualize the topology, replay the simulation, and analyze the network performance. A screenshot of the scenario loaded in NetSimulyzer is shown below.

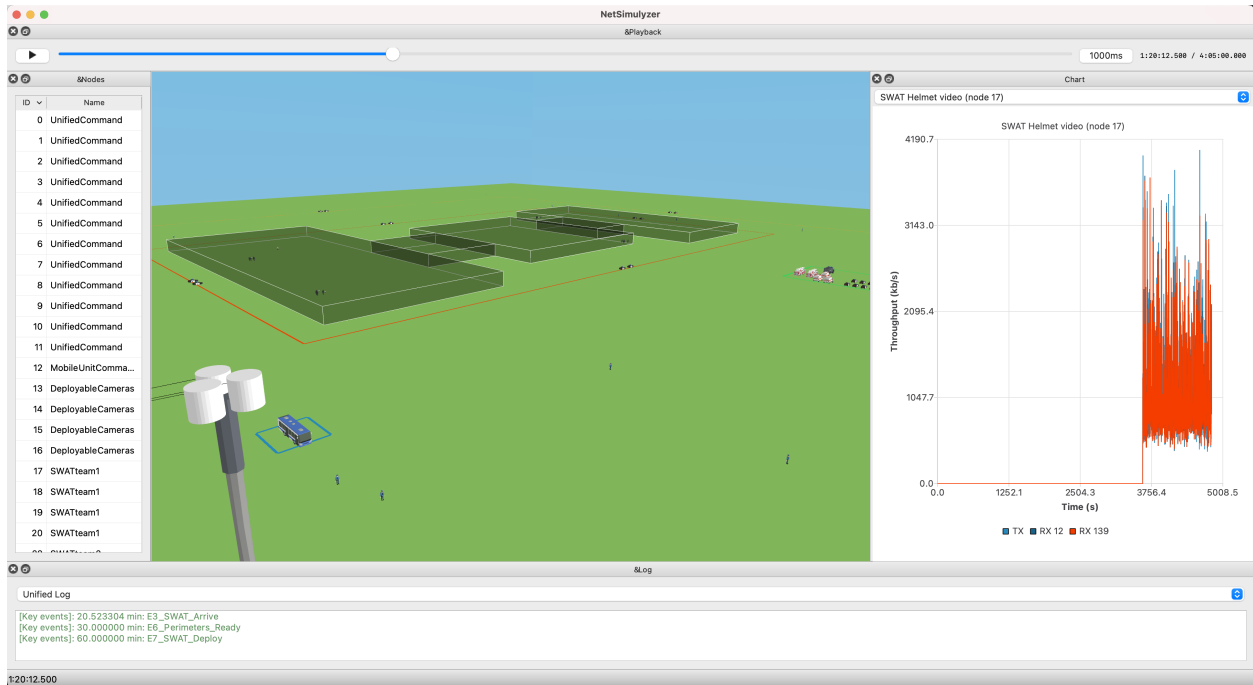


Fig. 1: Screenshot of school shooting scenario in NetSimulyzer

### 8.2.5 Extending the scenario

Even though several attributes have been exposed to change the network parameters, the physical deployment remains the same. Using the proposed design, researchers interested in evaluating different technologies (e.g., NR) or variations of the sample LTE deployment can simply extend the class `ns3::psc::SchoolShootingNetworkTechnologyHelper`. For visualization, only a small portion of the `SchoolShootingLteVisHelper` is actually technology dependent (e.g., accessing the eNodeBs), therefore the helper can easily be reused.

## BIBLIOGRAPHY

- [TS22179] 3GPP TS 22.179 “Mission Critical Push To Talk (MCPTT) Mission Critical Push to Talk (MCPTT) over LTE; Stage 1”
- [TS24379] 3GPP TS 24.379 “Mission Critical Push To Talk (MCPTT) call control; Protocol specification”
- [TS24380] 3GPP TS 24.380 “Mission Critical Push To Talk (MCPTT) media plane control; Protocol specification”
- [TS365792] 3GPP TS 36.579-2 “Mission Critical Push To Talk (MCPTT) User Equipment (UE) Protocol conformance specification”
- [NIST.IR.8206] Frey, J., Pieper, J., and Thompson, T., “Mission Critical Voice QoE Mouth-to-Ear Latency Measurement Methods”, February 2018.
- [NIST.IR.8236] Varin, P., Sun, Y., and Garey, W., “Test Scenarios for Mission Critical Push-To-Talk (MCPTT) Off-Network Mode Protocols Implementation”, October 2018.
- [NIST2016] Rouil, R., Cintrón, F.J., Ben Mosbah, A. and Gamboa, S., “An LTE Device-to-Device module for ns-3”, in Proceedings of the Workshop on ns-3, 15-16 June 2016, Seattle (Washington).
- [NIST2017] Rouil, R., Cintrón, F.J., Ben Mosbah, A. and Gamboa, S., “Implementation and Validation of an LTE D2D Model for ns-3”, in Proceedings of the Workshop on ns-3, 13-14 June 2017, Porto (Portugal).
- [NIST2019] Gamboa, S., Thanigaivel, R. and Rouil, R., “System Level Evaluation of UE-to-Network Relays in D2D-enabled LTE Networks”, in Proceedings of the 2019 IEEE 24th International Workshop on Computer Aided Modeling and Design of Communication Links and Networks, 11-13 September 2019, Limassol (Cyprus).
- [NIST2019b] Rouil, R., Liu, C., Izquierdo Manzanares, A., “Guidelines for Generating Public Safety Benchmark Scenario Set”, NIST Interagency/Internal Report (NISTIR) - 8247, May 2019.
- [NIST2021] Garey, W., Henderson, T.R., Sun, Y., Rouil, R. and Gamboa, S., “Modeling MCPTT and User Behavior in ns-3”, in Proceedings of the 11th International Conference on Simulation and Modeling Methodologies, Technologies and Applications (Simultech 2021), 7-9 July 2021, Paris (France).
- [NIST2021b] Black, E., Gamboa, S. and Rouil, R., “NetSimulyzer: a 3D Network Simulation Analyzer for ns-3”, in Proceedings of the Workshop on ns-3, 23-25 June 2021, Virtual Event (USA).
- [NIST2021c] Rouil, R., Izquierdo Manzanares, A., Liu, C. and Garey, W., “Modeling Public Safety Communication Scenarios: School Shooting Incident”, NIST Technical Note (NIST TN) - 2186, October 2021.